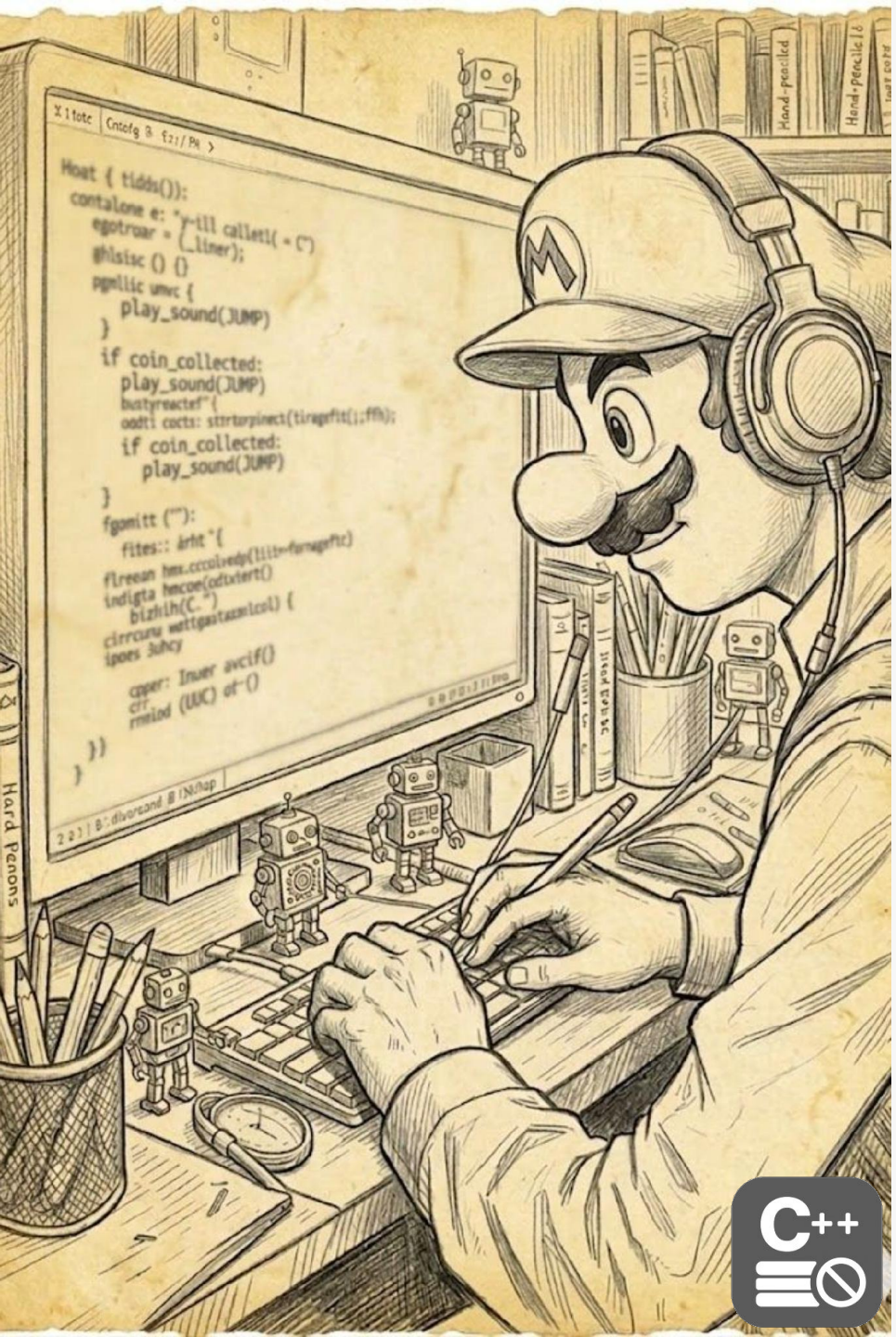
Interesting Allocators in Games



Manual Memory Management

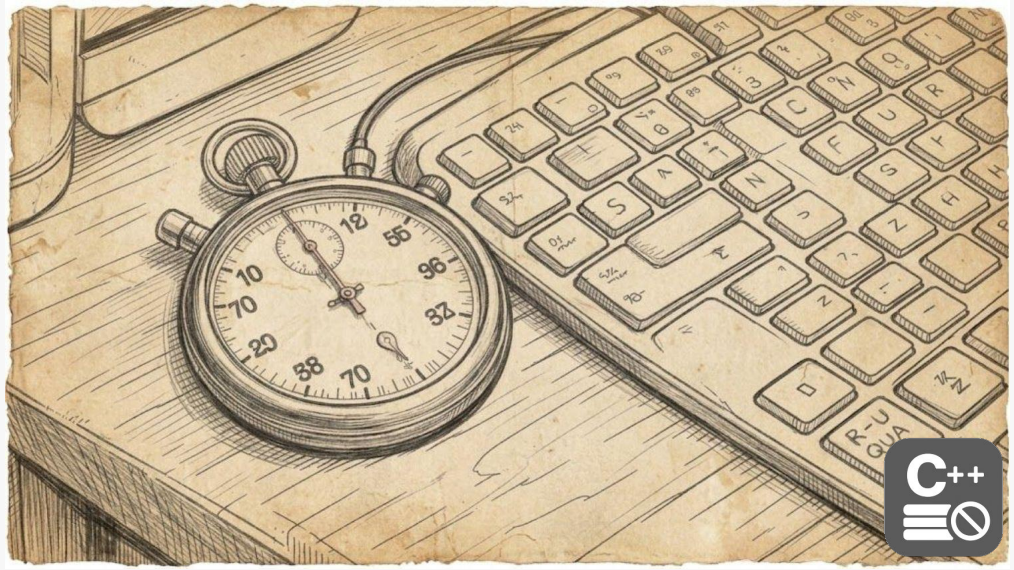
A well-known topic but worth revisiting.

In C++, memory is managed manually via new/delete or RAII objects. There is no garbage collector running in the background.

Lifetime Control

We fully control when memory is allocated and released. Unlike GC languages, there is no "background magic" — allocations happen exactly where we write them.

Deterministic Performance



Allocation and deallocation happens exactly where they are called.

No surprises, you know the cost and control the timing.

This is critical for real-time systems.

A garbage collector can suddenly trigger a stop-the-world pause for milliseconds and completely unacceptable in a game loop running at 60+ FPS.

Maximum Control Over Resources



Custom Allocators

Write your own allocators, memory pools, and arenas — optimized for specific tasks and data patterns.



Deferred Deletion

Delay deallocation until it's safe

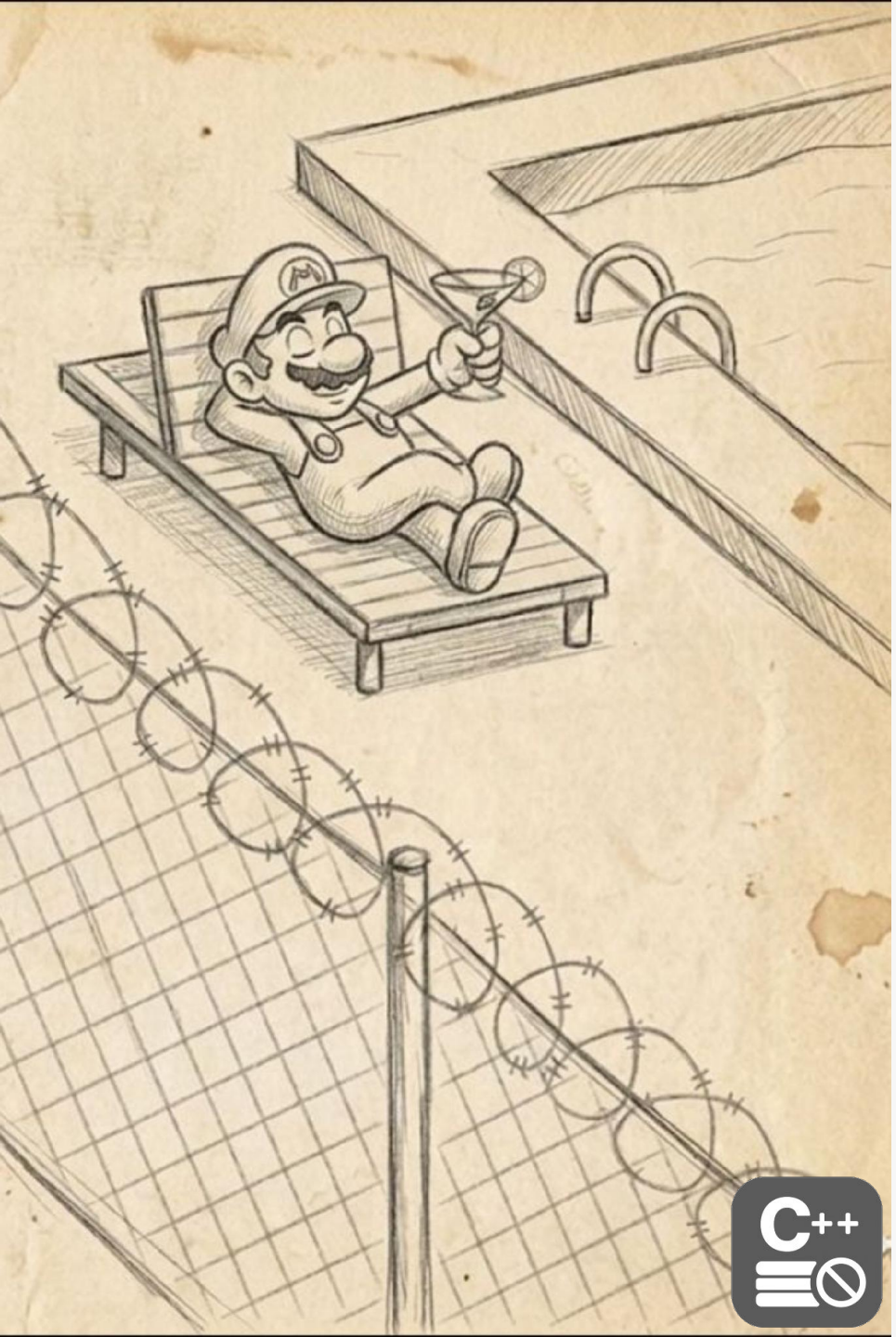
Avoiding use-after-free without paying for reference counting.



Ring Buffers

Pre-allocated circular buffers for event queues and transient data

Zero allocation at runtime.



General-Purpose Allocators

"The standard allocator works against you"

Game Engine Architecture (Jason Gregory)

Standard Allocator

General-purpose allocators are designed for wide usage, not performance.

Loss of Timing Predictability

General allocators are optimized for a wide scenarios

Not real-time constraints.

Allocation latency varies unpredictably.

Memory Fragmentation

Universal allocator is flexible

Always leads to "holes" in the heap

Frequently allocating and deallocating blocks slowing heap.



More Hidden Costs

Excessive Metadata Overhead

Standard allocator stores size, reference counts, and bookkeeping data for every single block.
At thousands of allocations per frame, this overhead compounds significantly.

No Control Over Data Layout

A general-purpose allocator has no knowledge of your data's lifecycle.
Frame-temporary objects and level resources are treated identically wasting both performance and locality.

You Pay for Flexibility You Don't Need

Game engines have structured, predictable memory patterns.

The standard allocator assumes unpredictable workloads.

This mismatch is actively harmful to determinism, cache behavior, and frame-time stability.

→ Frame-scoped allocations (particles, effects)

→ Level-scoped bulk loads (assets, geometry)

→ Fixed-pool objects (bullets, AI nodes)

Which Pac-Man Corrupted the Memory?

Not all strings are equal. Each declaration carries different allocation semantics — some silent, some costly.

```
const char* pacman1 = "pac-man";
                                                    ???
std::string pacman2 = "pac-man";
                                                    ???
eastl::string pacman3 = "pac-man";
                                                    ???
frozen::string<16> pacman4 = "pac-man";
                                                    ???
const char[] pacman5 = "pac-man";
                                                    ???
```

Which Pac-Man Corrupted the Memory?

Not all strings are equal. Each declaration carries different allocation semantics — some silent, some costly.

`const char* pacman1 = "pac-man";`	Static segment. Zero runtime allocation.
`std::string pacman2 = "pac-man";`	???
`eastl::string pacman3 = "pac-man";`	???
`frozen::string<16> pacman4 = "pac-man";`	???
`const char[] pacman5 = "pac-man";`	???

Which Pac-Man Corrupted the Memory?

Not all strings are equal. Each declaration carries different allocation semantics — some silent, some costly.

`const char* pacman1 = "pac-man";`

Static segment. Zero runtime allocation.

`std::string pacman2 = "pac-man";  <- bad guy`

Heap allocation via system allocator. Potential SSO bypass.

`eastl::string pacman3 = "pac-man";`

???

`frozen::string<16> pacman4 = "pac-man";`

???

`const char[] pacman5 = "pac-man";`

???

Which Pac-Man Corrupted the Memory?

Not all strings are equal. Each declaration carries different allocation semantics — some silent, some costly.

const char* pacman1 = "pac-man";

Static segment. Zero runtime allocation.

std::string pacman2 = "pac-man"; <- bad guy

Heap allocation via system allocator. Potential SSO bypass.

eastl::string pacman3 = "pac-man"; <- not good guy

EA STL — supports custom allocators, but default still hits heap.

frozen::string<16> pacman4 = "pac-man";

???

const char[] pacman5 = "pac-man";

???

Which Pac-Man Corrupted the Memory?

Not all strings are equal. Each declaration carries different allocation semantics — some silent, some costly.

const char* pacman1 = "pac-man";

Static segment. Zero runtime allocation.

std::string pacman2 = "pac-man"; <- bad guy

Heap allocation via system allocator. Potential SSO bypass.

eastl::string pacman3 = "pac-man"; <- not good guy

EA STL — supports custom allocators, but default still hits heap.

frozen::string<16> pacman4 = "pac-man";

Stack-allocated, fixed capacity. Zero heap traffic.

const char[] pacman5 = "pac-man";

???

Which Pac-Man Corrupted the Memory?

Not all strings are equal. Each declaration carries different allocation semantics — some silent, some costly.

```
const char* pacman1 = "pac-man";
```

Static segment. Zero runtime allocation.

```
std::string pacman2 = "pac-man"; <- 90% projects used std::strings
```

Heap allocation via system allocator. Potential SSO bypass.

```
eastl::string pacman3 = "pac-man"; <- not good guy
```

EA STL — supports custom allocators, but default still hits heap.

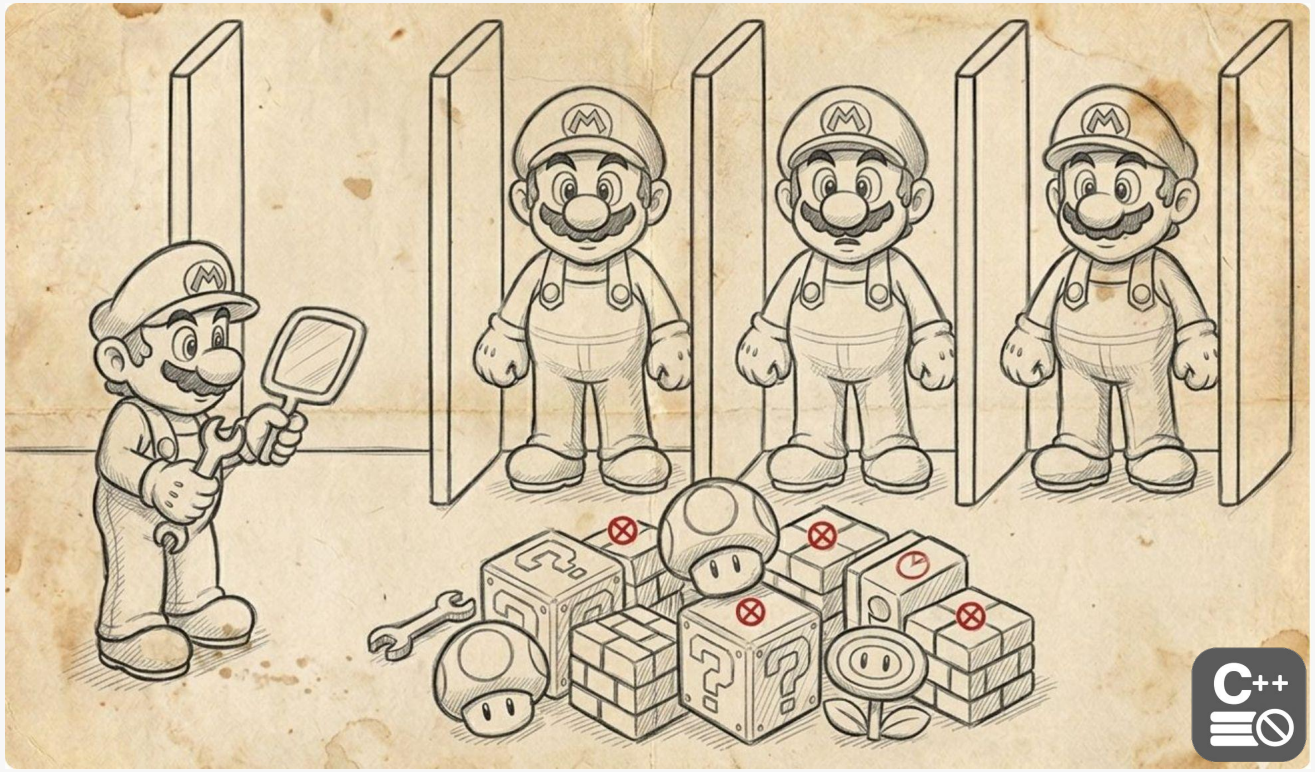
```
frozen::string<16> pacman4 = "pac-man";
```

Stack-allocated, fixed capacity. Zero heap traffic.

```
const char[] pacman5 = "pac-man";
```

Stack array, size deduced. No indirection, no heap.

Do you need your own allocator?



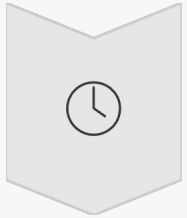
Trust the System

Code is correct, comfort and works, but slow

Follow the white rabbit…

Full control, but you own every bug, leak, and edge case.

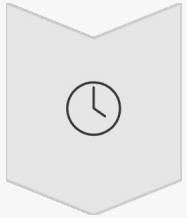
When You Need a Custom Allocator



???

???

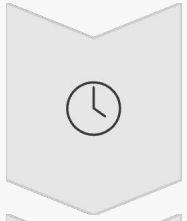
When You Need a Custom Allocator



Allocation Timing Control

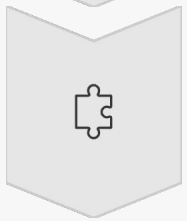
Prevent accidental pauses from fragmentation or slow deallocations. Frame budgets demand deterministic behavior.

When You Need a Custom Allocator



Allocation Timing Control

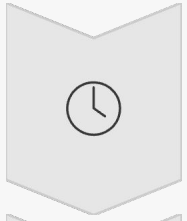
Prevent accidental pauses from fragmentation or slow deallocations. Frame budgets demand deterministic behavior.



Avoid Fragmentation

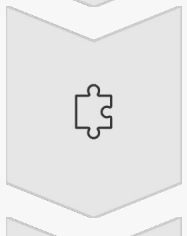
Frequent create/destroy of small objects (particles, bullets, effects) fragments the heap rapidly under a general allocator.

When You Need a Custom Allocator



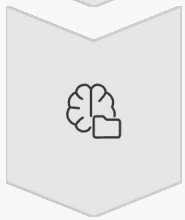
Allocation Timing Control

Prevent accidental pauses from fragmentation or slow deallocations. Frame budgets demand deterministic behavior.



Avoid Fragmentation

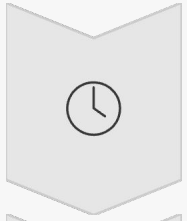
Frequent create/destroy of small objects (particles, bullets, effects) fragments the heap rapidly under a general allocator.



Predictable Memory Usage

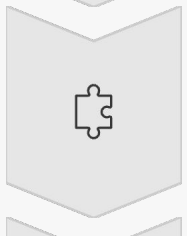
Consoles and mobile have strict limits. Pre-allocate a large arena (e.g., 64 MB) and sub-allocate within it.

When You Need a Custom Allocator



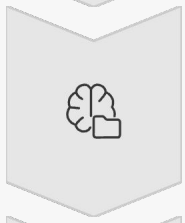
Allocation Timing Control

Prevent accidental pauses from fragmentation or slow deallocations. Frame budgets demand deterministic behavior.



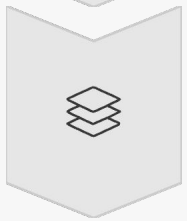
Avoid Fragmentation

Frequent create/destroy of small objects (particles, bullets, effects) fragments the heap rapidly under a general allocator.



Predictable Memory Usage

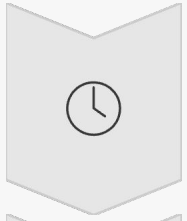
Consoles and mobile have strict limits. Pre-allocate a large arena (e.g., 64 MB) and sub-allocate within it.



Data Lifecycle Awareness

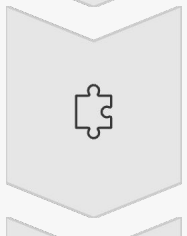
Arena allocators for frame-temporary objects; linear allocators for level resources loaded and freed as a single block.

When You Need a Custom Allocator



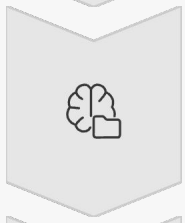
Allocation Timing Control

Prevent accidental pauses from fragmentation or slow deallocations. Frame budgets demand deterministic behavior.



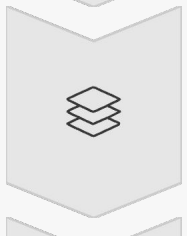
Avoid Fragmentation

Frequent create/destroy of small objects (particles, bullets, effects) fragments the heap rapidly under a general allocator.



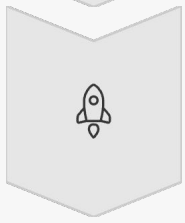
Predictable Memory Usage

Consoles and mobile have strict limits. Pre-allocate a large arena (e.g., 64 MB) and sub-allocate within it.



Data Lifecycle Awareness

Arena allocators for frame-temporary objects; linear allocators for level resources loaded and freed as a single block.



Maximum Performance

Control exact memory layout — merge same-type entities, avoid cache line pollution from "noisy" structures.

How Objects Laid in Memory?

If we allocate several objects sequentially, how does they will be laid in the heap?

```
new Obj_A; new Obj_B; new Obj_C; new Obj_D;
```

You might expect them to be contiguous.

But the allocator makes independent decisions for each request

How Are Objects Laid Out in Memory?

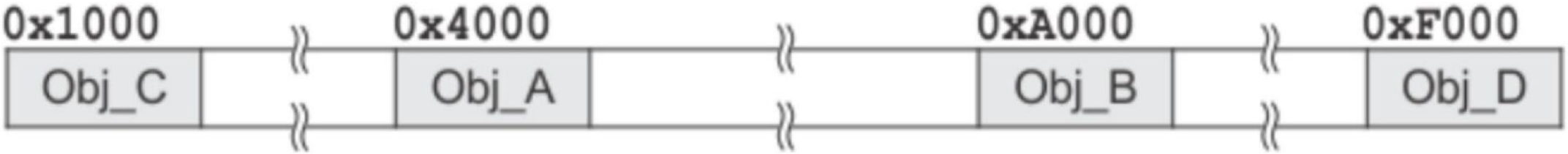
If we allocate several objects sequentially, how does they will be laid in the heap?

```
new Obj_A; new Obj_B; new Obj_C; new Obj_D;
```

You might expect them to be contiguous.

But the allocator makes independent decisions for each request

Reality of Heap Layout



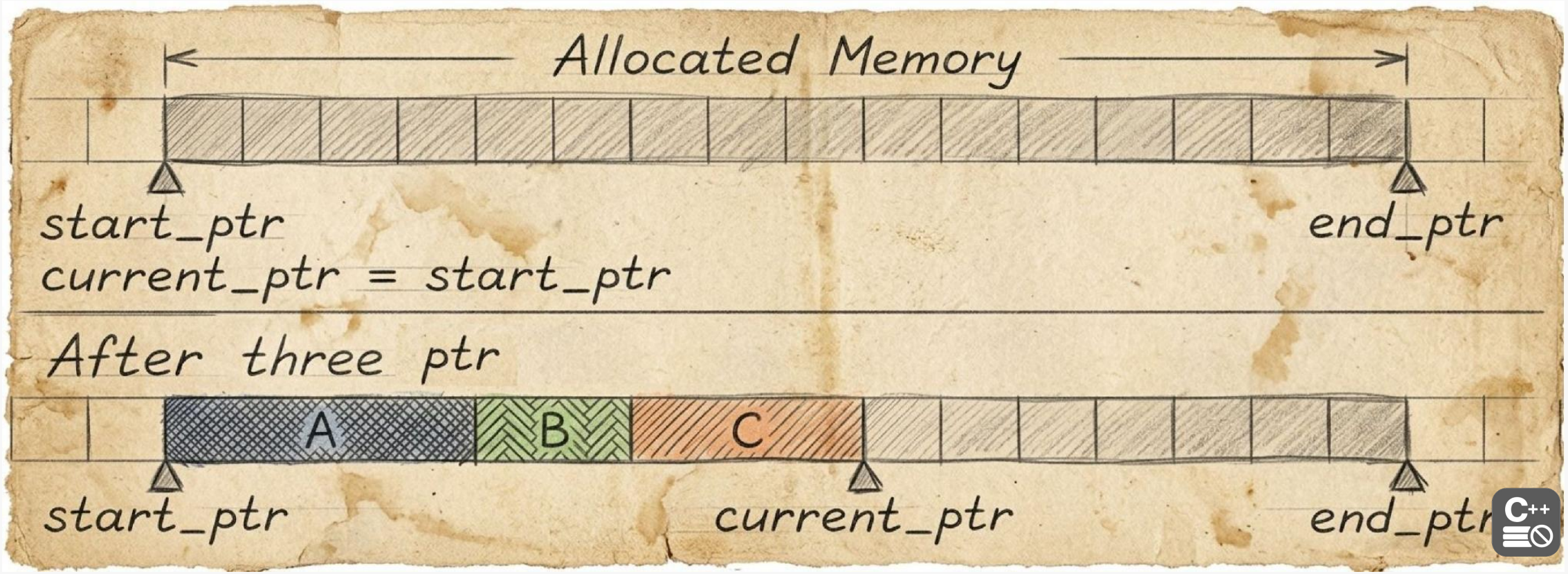
Each new call is an independent heap request.

Objects may be placed across non-contiguous memory regions, break cache locality and performance.

Theory is over…

Linear Allocator (Gold in game dev)

The simplest and most efficient memory-manager for game developers. Zero fragmentation, zero deallocation cost, and lightspeed-fast allocation. Use it where you need high performance and minimal overhead.



Where Linear Allocators Uses

```
char stack_buffer[1024];

buffer_resource stack_resource
 {stack_buffer, sizeof(stack_buffer)};

std::pmr::vector v{&stack_resource};

for (int i = 0; i < 100; ++i)
  v.push_back(i); // zero heap allocs
```

???

Where Linear Allocators Uses

```
char stack_buffer[1024];

buffer_resource stack_resource
 {stack_buffer, sizeof(stack_buffer)};

std::pmr::vector v{&stack_resource};

for (int i = 0; i < 100; ++i)
  v.push_back(i); // zero heap allocs
```

Frame Temporaries

Extremely fast

No deallocation needed.

Objects must not outlive the allocator.

???

Where Linear Allocators Uses

```
char stack_buffer[1024];

buffer_resource stack_resource
 {stack_buffer, sizeof(stack_buffer)};

std::pmr::vector v{&stack_resource};

for (int i = 0; i < 100; ++i)
  v.push_back(i); // zero heap allocs
```

Frame Temporaries

Extremely fast

No deallocation needed.

Objects must not outlive the allocator.

Level Statics

Level-scoped objects and destructible data

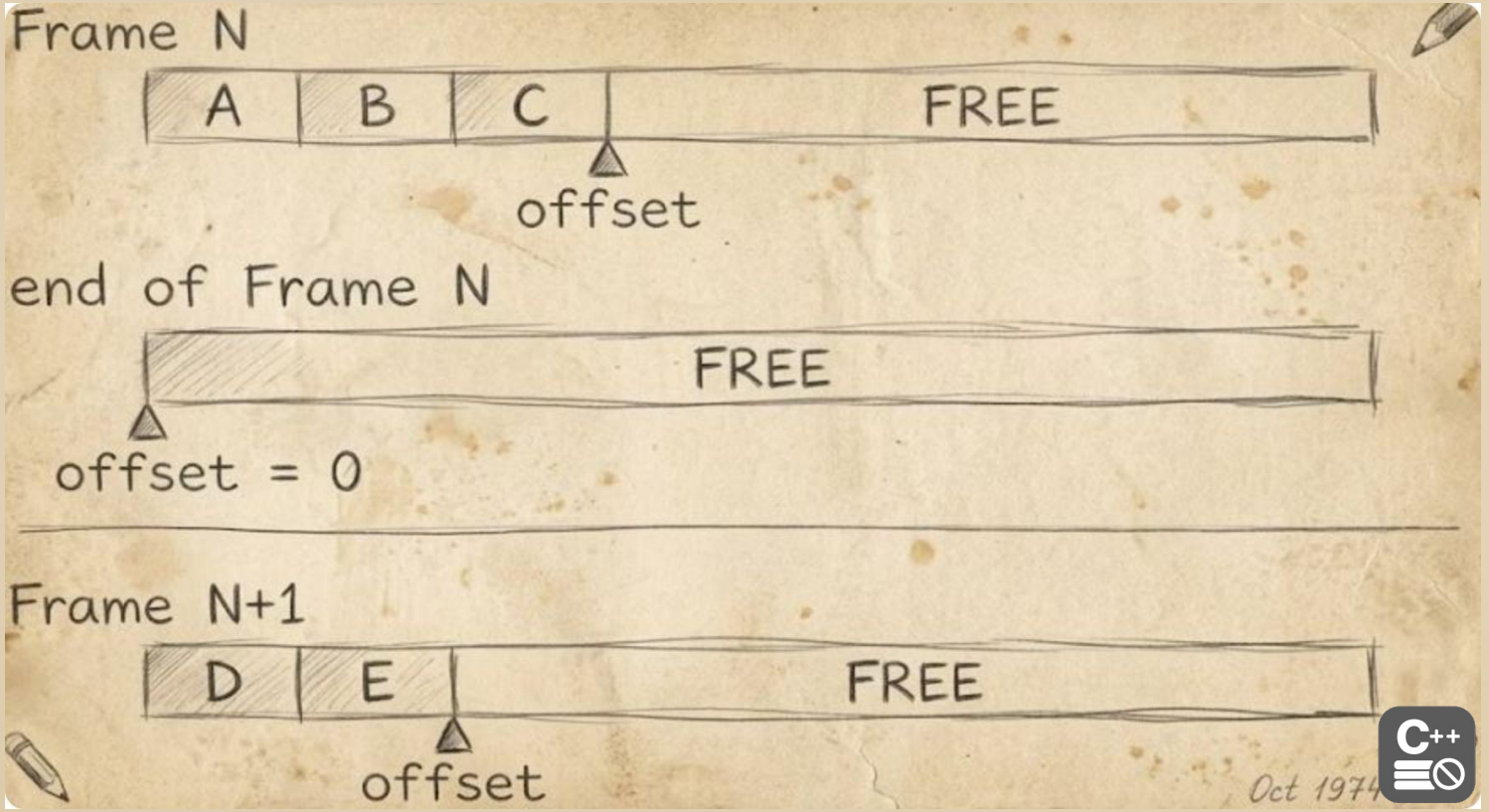
Unloaded in a single bulk operation on exit.

Frame Allocator

A specialized type of linear allocator for time-bounded workloads.
Memory is allocated freely within a frame, then the entire region is reset in O(1) at frame boundary.

No individual frees

No fragmentation.



Frame Allocator: Use Cases

Rendering

Command buffers, shader uniform data, and debug geometry — all discarded after GPU submission.

Event Queues

User input, keyboard, and controller events. Consumed within the frame, never persisted.

AI Systems

Pathfinding scratch data, NPC behavior trees, and reaction logic — all stale by the next frame.

Budget ~1.5× actual usage to absorb per-frame spikes without reallocation.

Thread-Local Frame Allocator



A frame allocator scoped to a single thread or task eliminates synchronization overhead entirely.

Each worker owns its memory arena no atomics, no locks, no contention.

Zero Synchronization
No atomics, no mutexes — pure sequential access.

Strictly per-thread;
memory must not cross thread boundaries.

???
???

Thread-Local Frame Allocator



A frame allocator scoped to a single thread or task eliminates synchronization overhead entirely.

Each worker owns its memory arena no atomics, no locks, no contention.

Zero Synchronization
No atomics, no mutexes — pure sequential access.

Strictly per-thread;
memory must not cross thread boundaries.

2× Budget Rule
Reserve headroom for per-frame allocation spikes.

Double-Buffered Frame Allocator

An evolution of the frame allocator that solves the producer-consumer timing problem.

Two independent memory regions alternate each frame

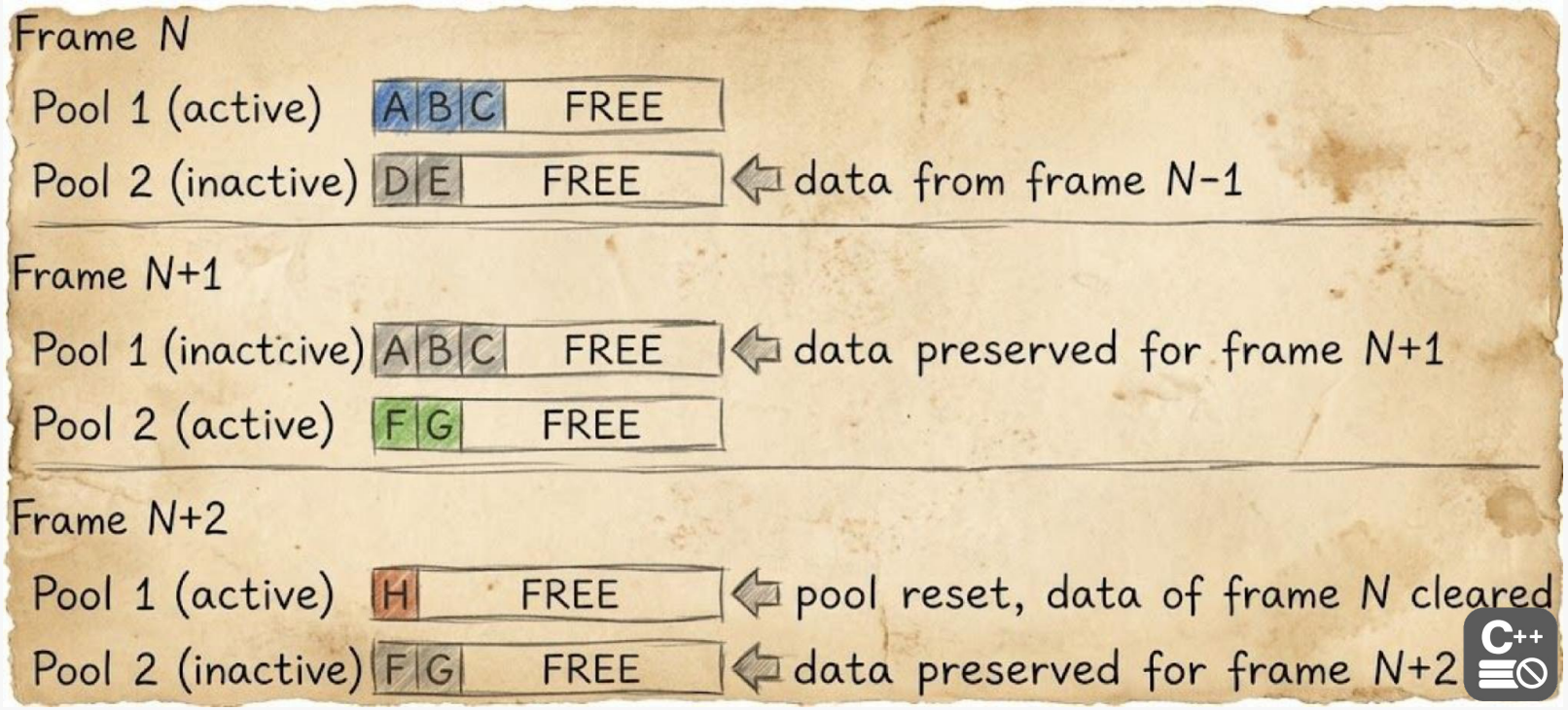
- while the GPU consumes frame N,
  - the CPU writes frame N+1 into the other buffer.

No stalls

no fences

no race conditions

>> very complex <<

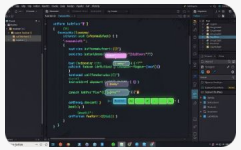


Double-Buffered Frame Allocator: Use Cases



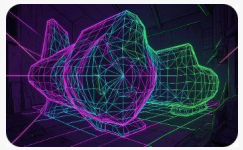
GPU Commands

CPU builds frame N+1 commands while GPU executes frame N. No stalls.



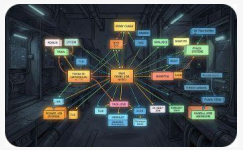
Uniform Buffers

Shader data stays valid until the GPU consumes it — no premature overwrites.



Vertex / Index Buffers

Previous-frame data remains accessible for streaming and async uploads.



Job System & AI

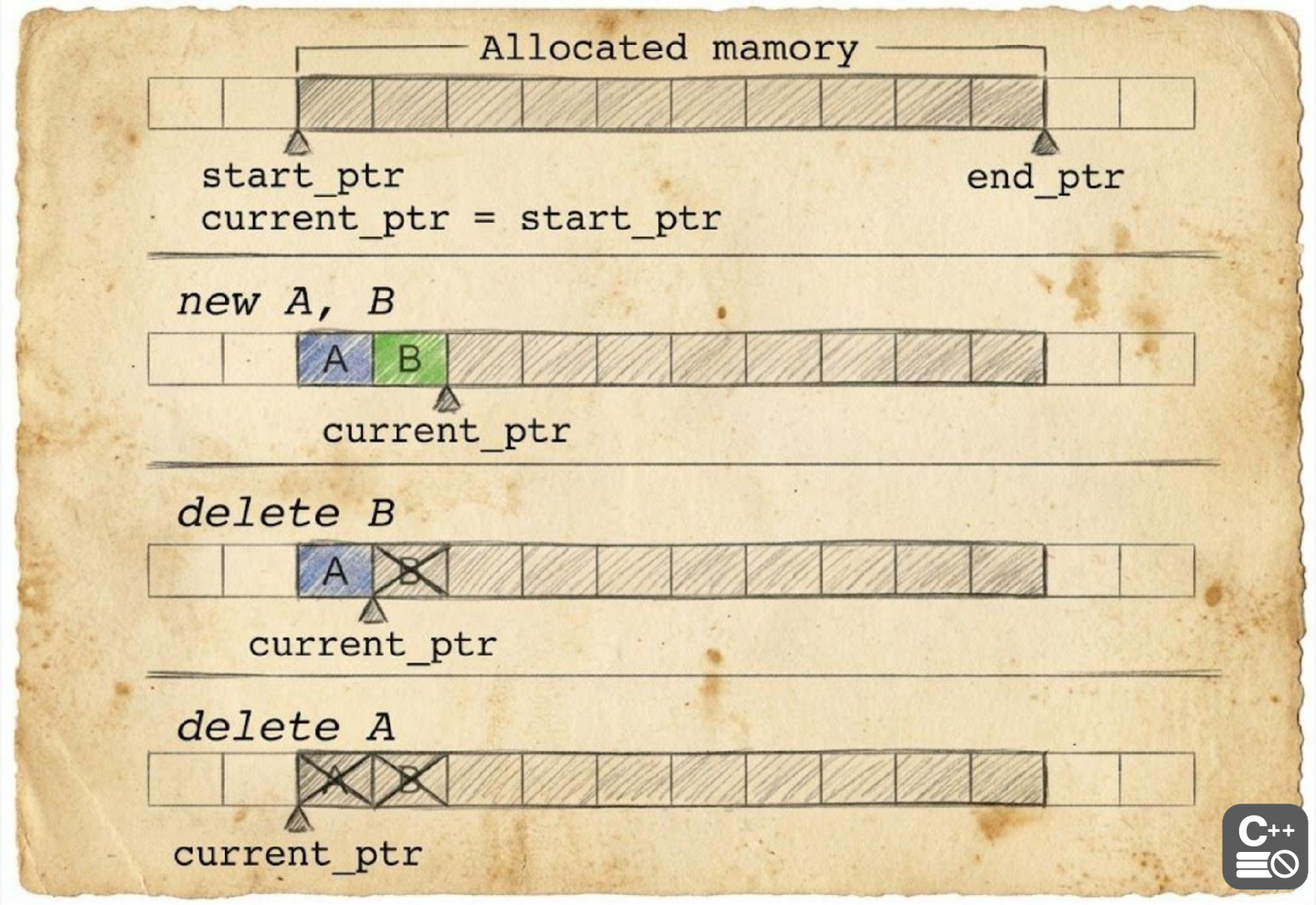
Prior-frame job results and committed AI state stay stable, avoiding nondeterminism.

Stack allocator (gamedev gold)

Simple and efficient approach speed, predictability and minimal overhead.

Memory is allocated and freed in strict LIFO order

The last allocated block of memory must be released first.



Stack allocator (use cases)

Rendering

Nested render passes

OpenGL/Vulkan state

Shader compilations

Scene Traversal

Depth-first traversal

Scene-tree navigation

Parsing & Loading

XML/JSON parsers

Resource loading

Pathfinding

Recursive algorithms A*,

JPS (Jump Point Search)

BucketNav (D*)

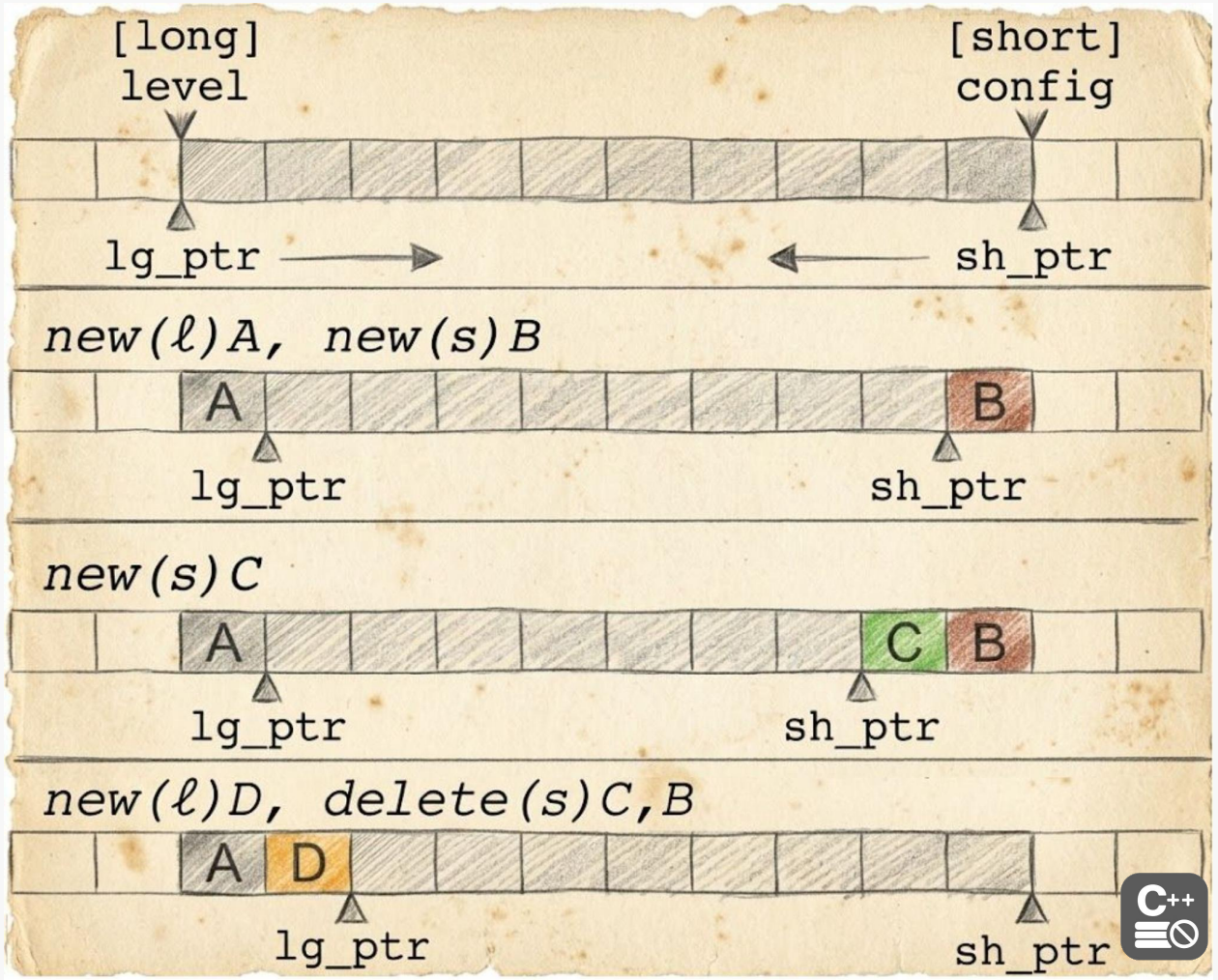


Double Stack Allocator

A powerful extension of the classic stack allocator.

Designed for workloads that load large volumes of mixed data with opposing lifetime patterns.

Double Stack Allocator (Lifecycle)



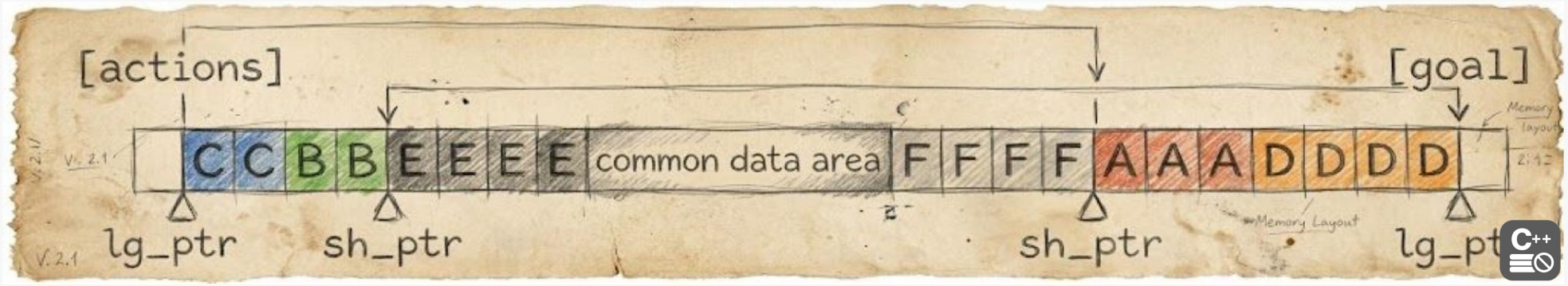
Can we improve Double Stack Allocator ?

Can we improve Double Stack Allocator ?

We can improve everything…

© John Carmack

Triple Stack Allocator

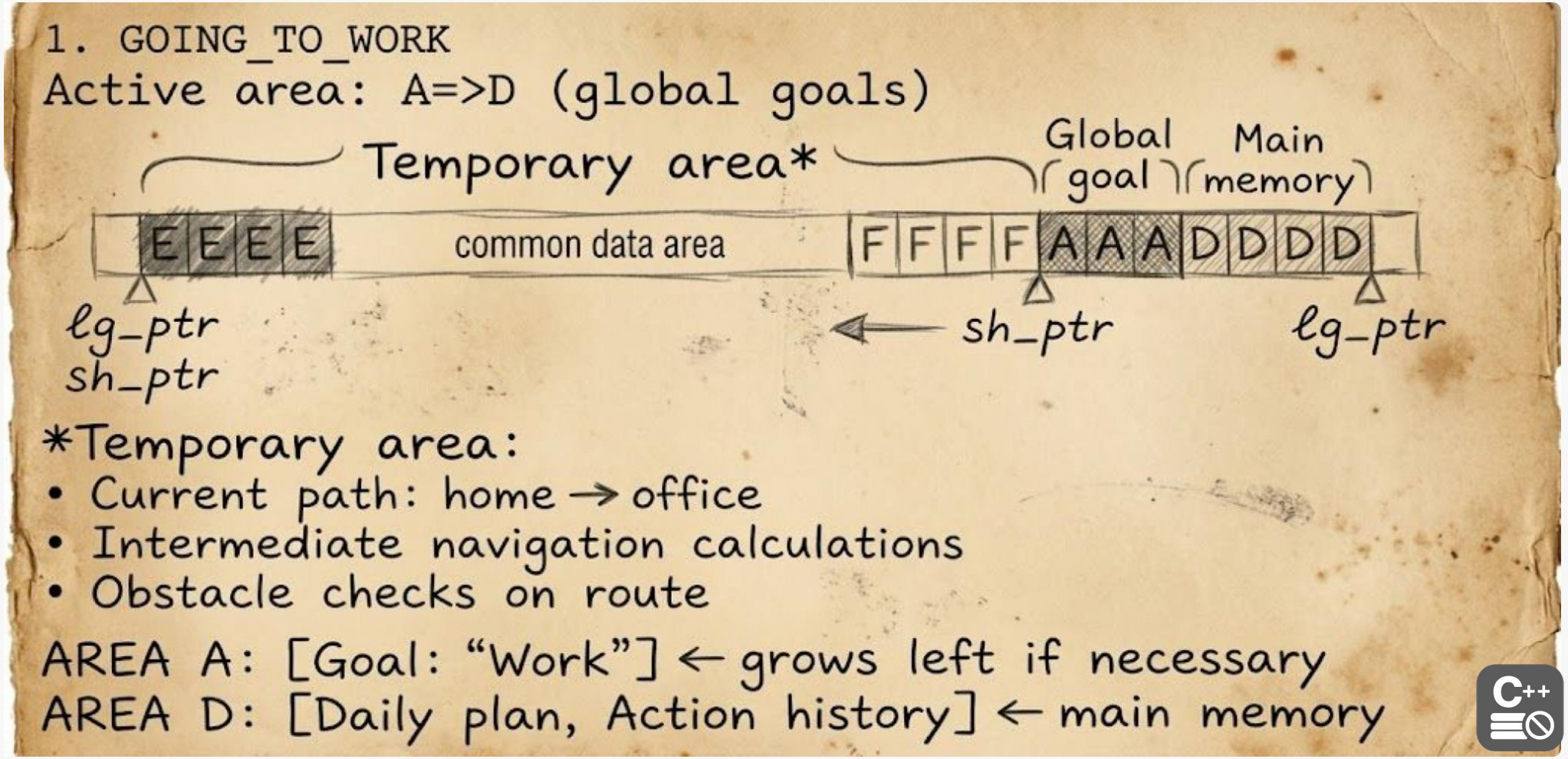


The peak of stack-allocator evolution.

The double-stack mechanism was extended to three growing regions, enabling efficient memory management for complex behavioral patterns.

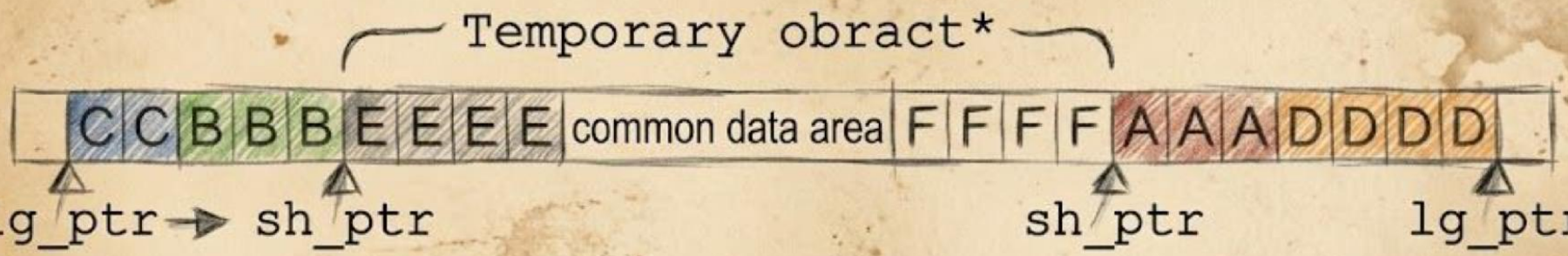
Used in the AI NPC architecture of The Sims series.

Triple Stack Allocator (Step 1)



Triple Stack Allocator (Step 2)

2. URGENT_NEED_BATHROOM
Switching to areas C=>A for procossing short target



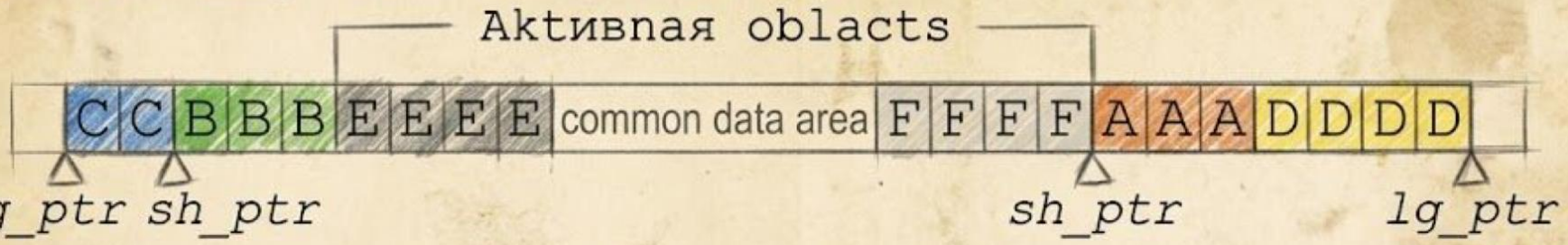
*Temporary area:
- Searching for nearest bathroom
- Interrupting current route
- Calculating time for biological need

AREA C: [Goal: "Bathroom", Priority: HIGH]<--grows right
AREA B: [Saving state of "going to work"]

Triple Stack Allocator (Step 3)

3. USING_BATHROOM

Crontoining work in Area E=>F



AREA E=>F:

[Local goal: "Toilet" - IN PROGRESS]:

- Animation of uses of toilet
- Timer execution dection
- Sim's need update Sim's (comfort +80)
- Check completion denctвuk

Frozen until local goal is complete:

AREAS C=>B, A=>D: [State of global goal for recovery]

Triple Stack: Where It's Used

Rare in modern games, coz we are have enough memory for all, I hope...

But where resources are constrained, the triple stack delivers **2× memory savings.**

Embedded & IoT

Microcontrollers with tight memory budgets

Automotive ECU

CAN processing, diagnostics, configuration

Industrial PLC

Input-signal processing, control logic, HMI

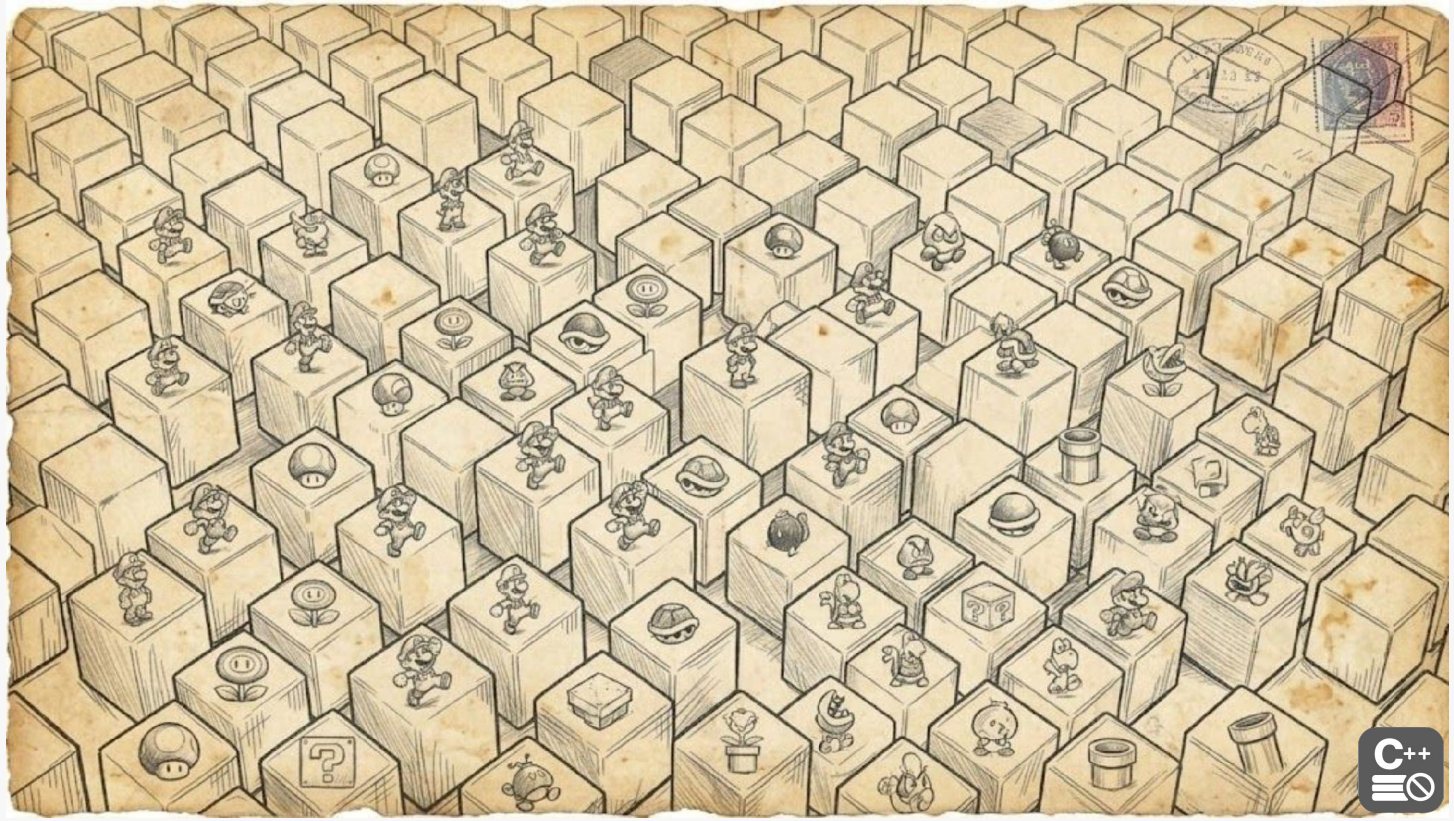
SCADA Systems

Real-time data buffering and archiving

Robotics

Sensor processing, motion planning, AI

Memory Pool



Linear and general-purpose allocators suffer from fragmentation and variable allocation cost.

A memory pool pre-allocates a contiguous array of fixed-size blocks, eliminating both.

O(1) Allocation

No search, no coalescing, just pop from the free list

Zero External Fragmentation

All blocks are identical, any object fits any free slot

Cache-Friendly

Contiguous memory improves spatial locality

Memory Pool

Block 0	Block 1	Block 2	Block 3	Block 4	Block 5	Block 6	Block 7
NPC_A	NPC_B	NPC_C	FREE	FREE	FREE	FREE	FREE

Free List: 3 → 4 → 5 → 6 → 7 → NULL

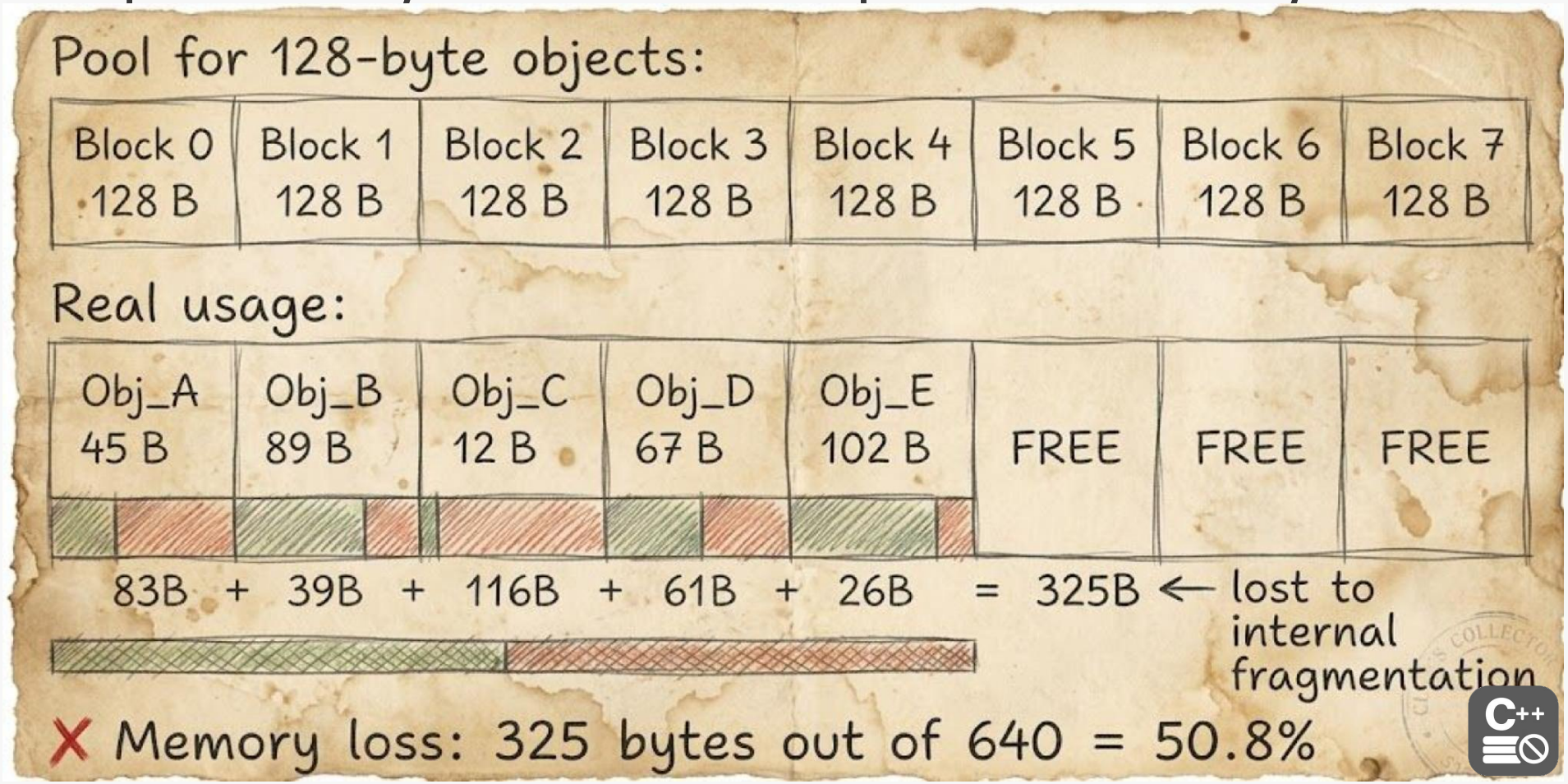
Used List: 0 → 1 → 2

```
NPC_A: { id: 001, pos: (10,20), health: 100, ai_state: PATROL }
NPC_B: { id: 002, pos: (15,25), health: 100, ai_state: IDLE }
NPC_C: { id: 003, pos: (8,18),  health: 100, ai_state: GUARD }
```



Memory Pool

It's important to know your data structures so the pool doesn't waste memory



Memory Pool: Use Cases

Best suited for high-frequency spawn/destroy cycles of **homogeneous objects.**



Particle Emitters

Sparks, smoke, fire, magic — bulk creation and destruction



Weather Systems

Raindrops, snowflakes, leaves in large quantities



Destruction Effects

Debris, decals, shards spawned on impact



Projectiles

Shells, arrows, spells — uniform size, high frequency



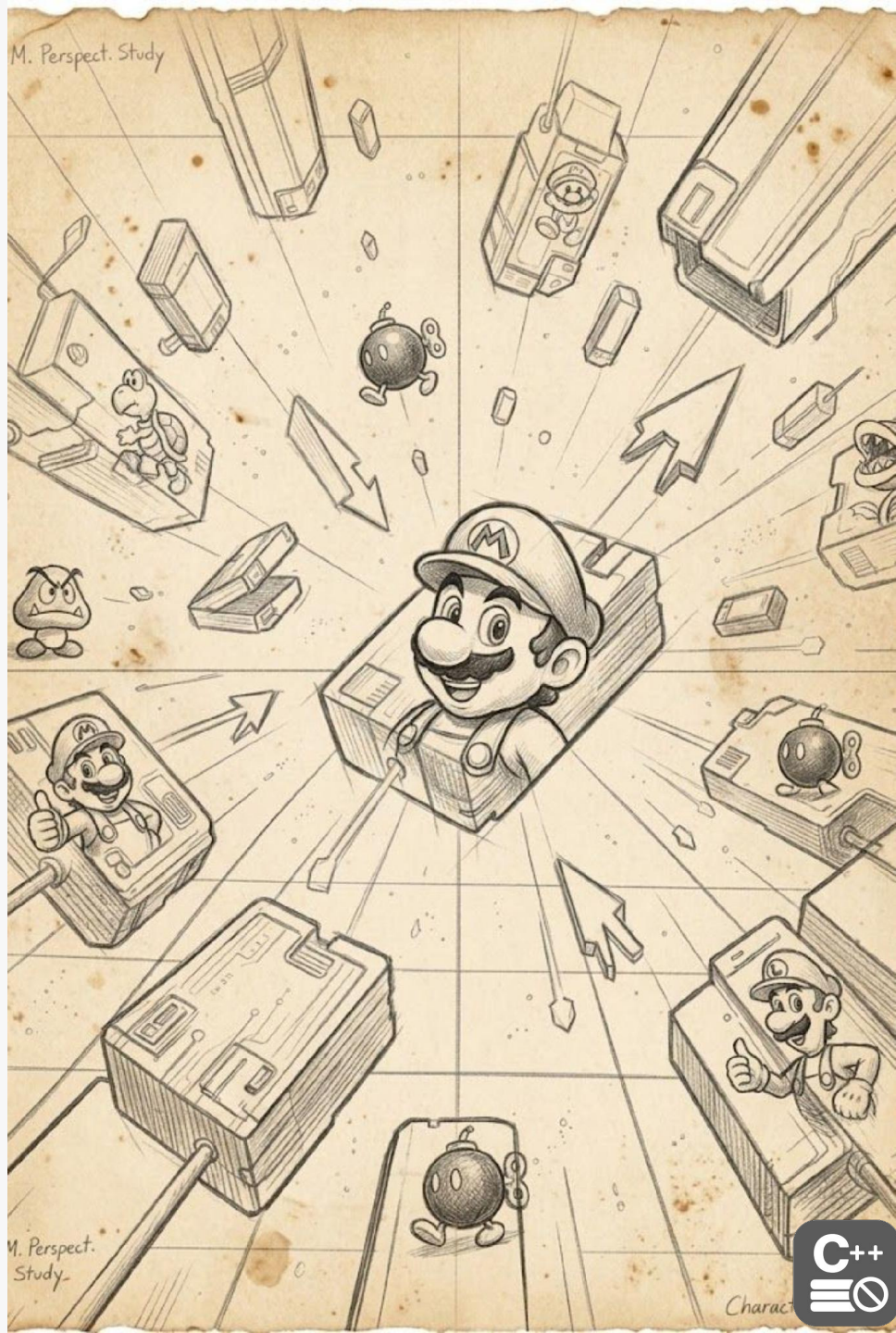
Pickups

Coins, bonuses, collectibles scattered across the world

Free-List Allocator

Memory pools deliver O(1) allocation and deallocation with minimal external fragmentation

All blocks are the same fixed size, so any free slot fits any new object.



Evolution of Free-List Allocator

1

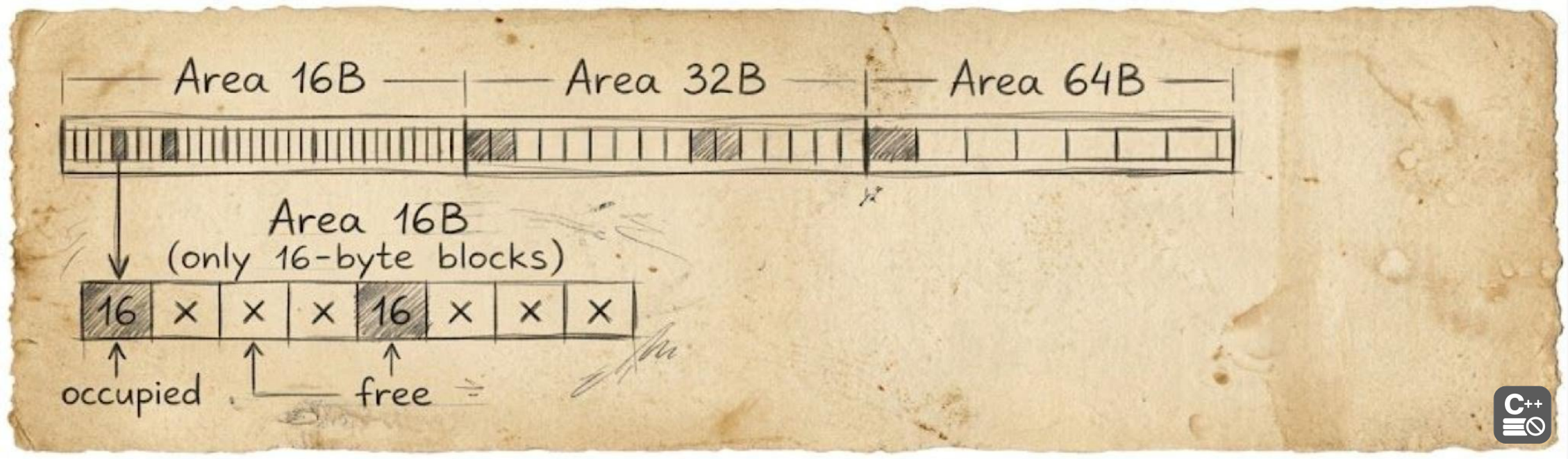
Free-List Pool
Single fixed block size, O(1) alloc/free

2

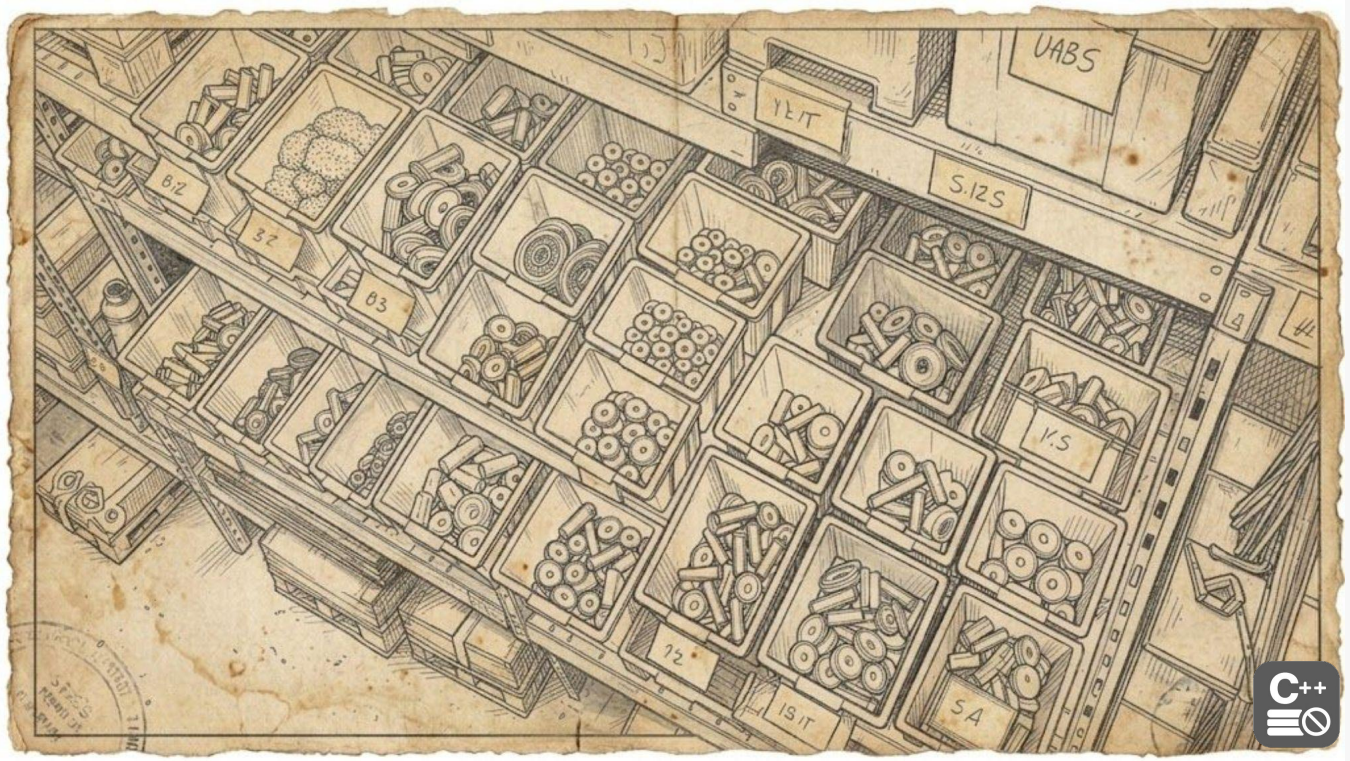
Segregated Free-List
Multiple size classes, O(1) within each class

3

Strict Segregated Free-List
Better fit for variable-size objects



Free Memory Split by Size Class

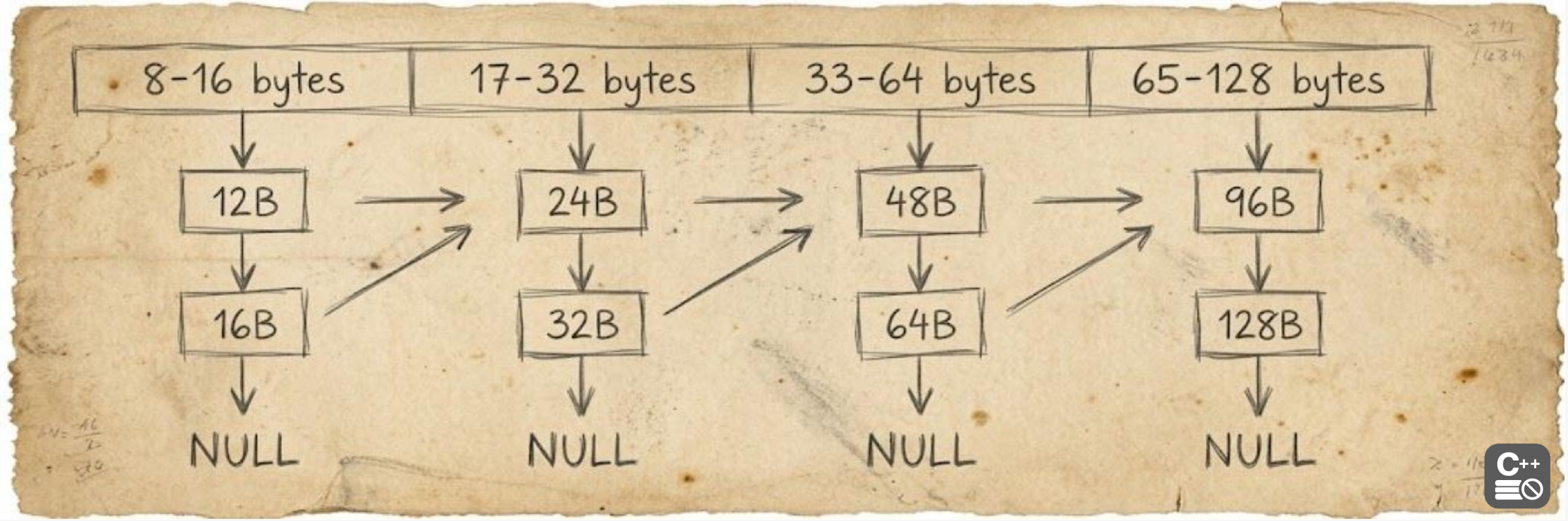


All free memory is partitioned into subsets (bins) based on block size. Each bin manages its own free list independently, so allocation becomes a simple bin lookup followed by a pop from that list.

Bin 0	Bin 1	Bin 2	Bin N
8–16 bytes	17–32 bytes	33–64 bytes	...and so on

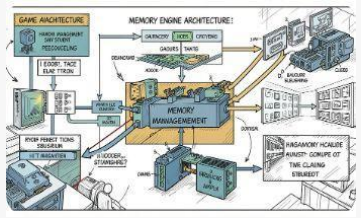
Each Subset Holds a Specific Size Range

Blocks are grouped into power-of-two size classes. When a request arrives, the allocator rounds up to the nearest class and pops from that bin is O(1) within the class.



Internal fragmentation is bounded: at most 50% waste per allocation, since each object fits within the next power-of-two class.

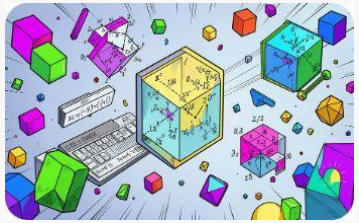
Segregated Free-Lists In Games



General-Purpose Heap

Engine's main allocator with size classes

(8, 16, 32, 64, 128 bytes).



Small Object Allocation

Math structures

Vector3, Matrix4x4, Quaternion

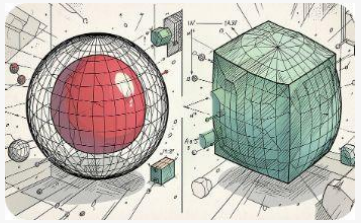
With predictable sizes.



String Management

Strings split into size classes

(16, 32, 64, 128, 256 chars).



Physics Primitives

Collision shapes

Sphere, Box, Capsule

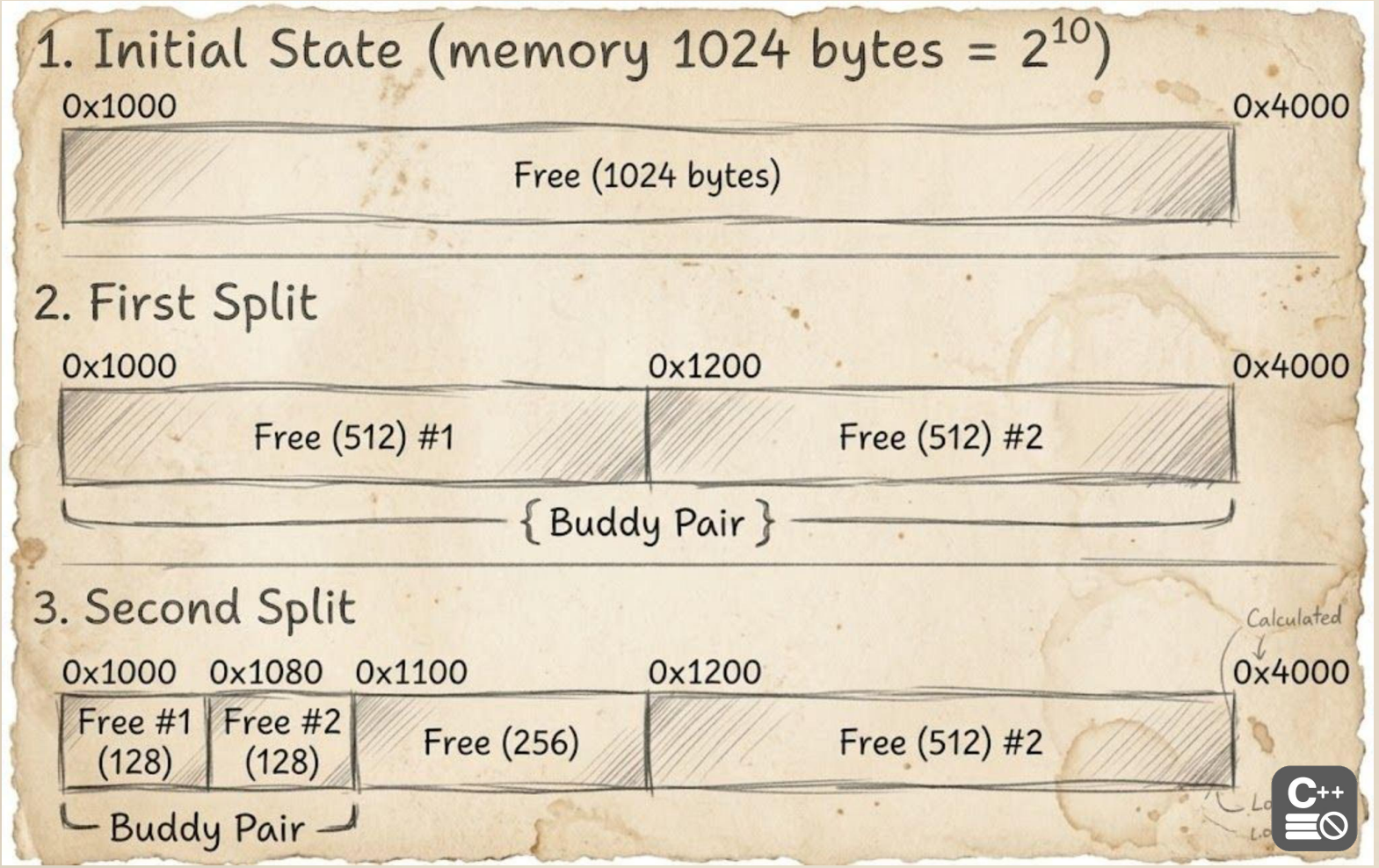
ConvexHull fit fixed size classes.

Buddy allocator

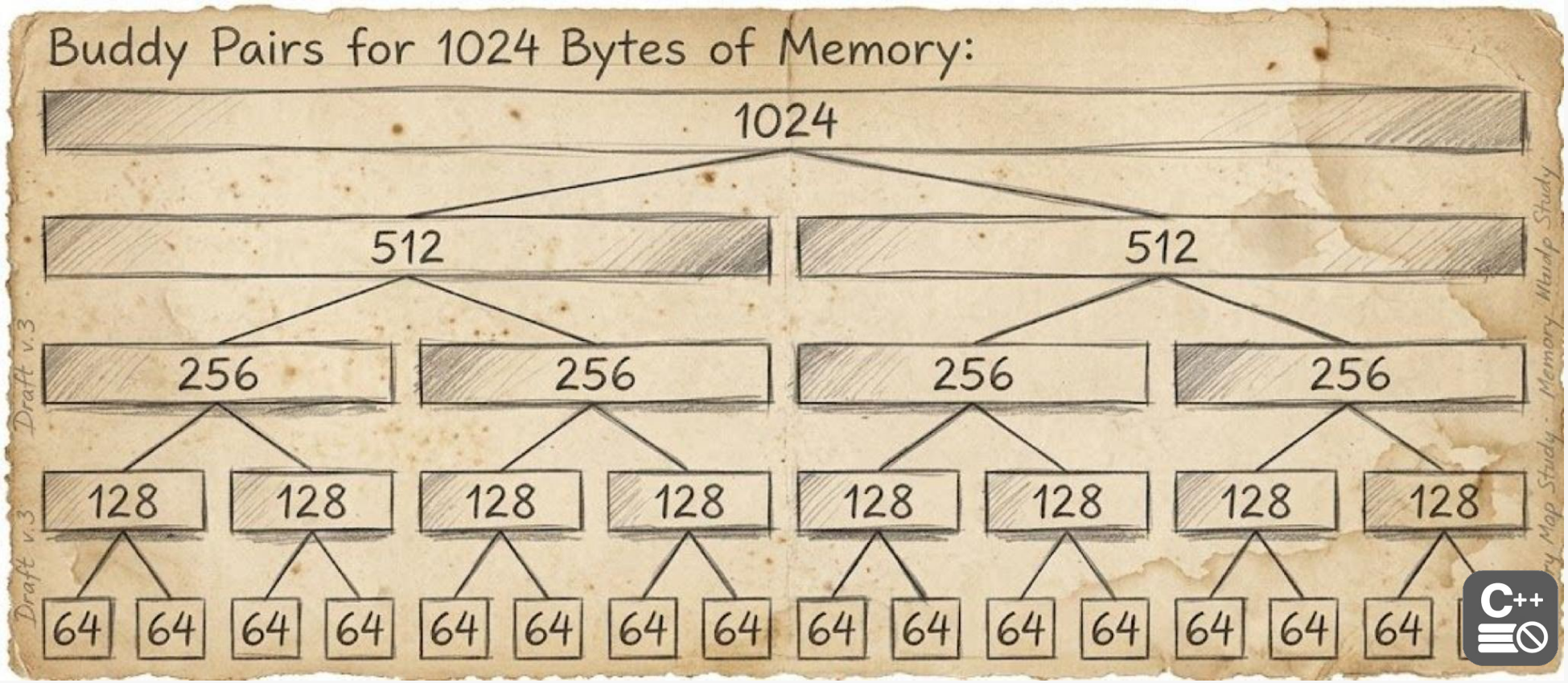
The buddy allocator divides memory by **fractally splitting blocks in half.**

Every allocated block is a power of two: 4, 8, 16, 32, 64, 128 bytes.

When a block is freed, its buddy is checked — if also free, they merge back, eliminating external fragmentation.



The Power-of-Two Invariant



1 **Start with 2ⁿ bytes**

The heap is initialized as one contiguous power-of-two block.

2 **Split until fit**

Blocks are halved recursively until the smallest power-of-two that fits the request.

3 **Merge on free**

Freed buddies coalesce back into larger blocks, restoring the original pool.

Buddy Allocator in Game AI Systems

The buddy allocator excelled in game AI — behavior trees, state machines, pathfinding agents, and decision trees all allocate local data blocks that fit power-of-two sizes naturally.



Behavior Trees

Local data blocks per node, power-of-two sized.



Finite State Machines

Per-state local memory with predictable sizes.



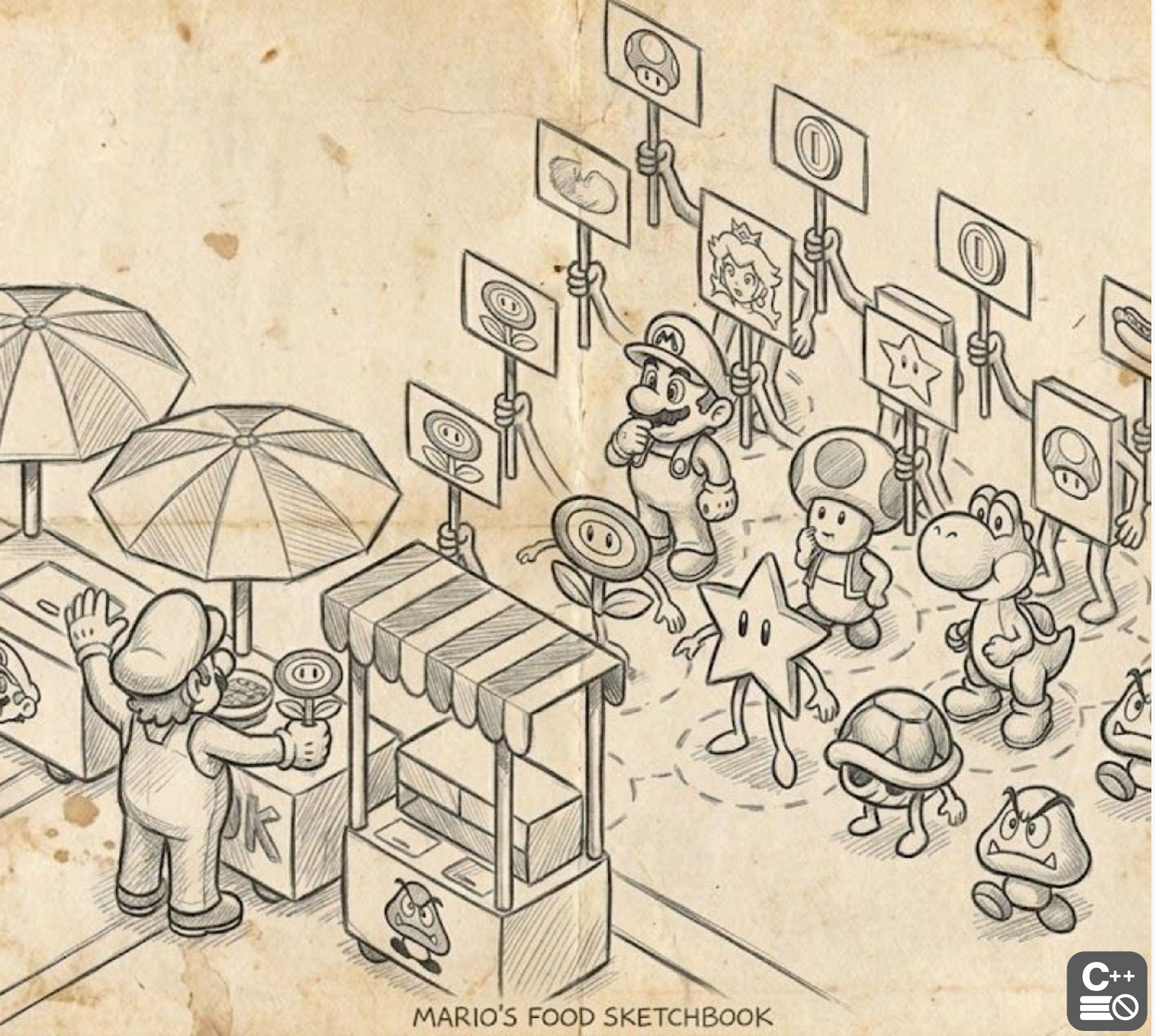
Pathfinding Agents

Navigation data packed to power-of-two boundaries.



Fuzzy Decision Trees

Local evaluation buffers with fixed-size allocations.



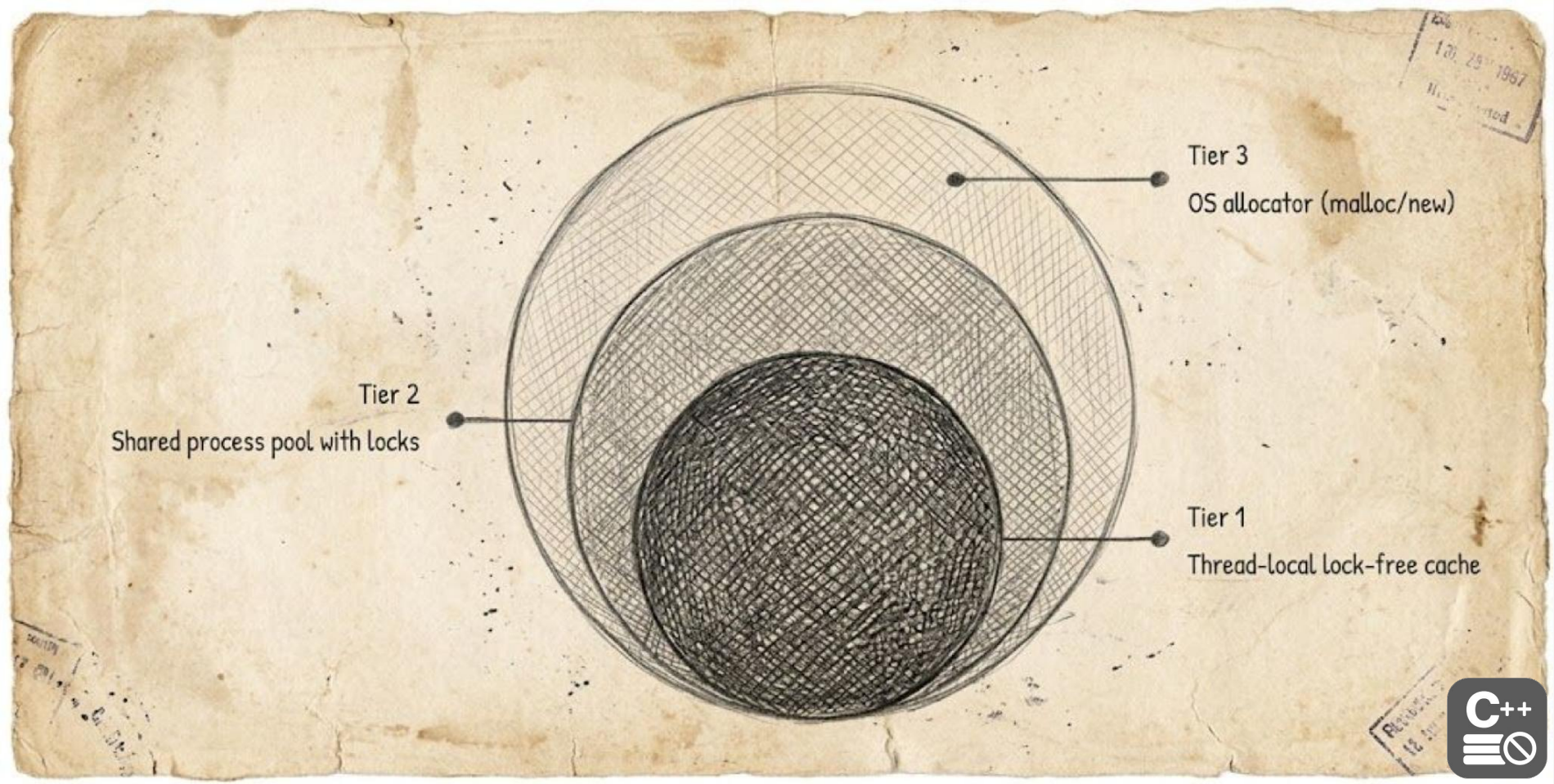
Thread-Local Cache Allocator

Standard `new`/`malloc` calls traverse the OS kernel on every request and contending with other processes, acquiring global locks, and suffering syscall overhead.

A thread-local cache allocator intercepts this path, keeping hot allocations entirely in user space.

Three-Tier Allocation Hierarchy

Memory requests are resolved at the lowest-cost tier available, falling through only when necessary.



Most allocations never leave Tier 1 — eliminating kernel transitions and cross-thread contention for the common case.

Why Thread-Local Allocators Wins

Lock-Free

Each thread owns its cache — zero atomic ops, zero mutexes on the hot path.

No Contention

Threads never compete for a shared heap. False sharing is eliminated by design.

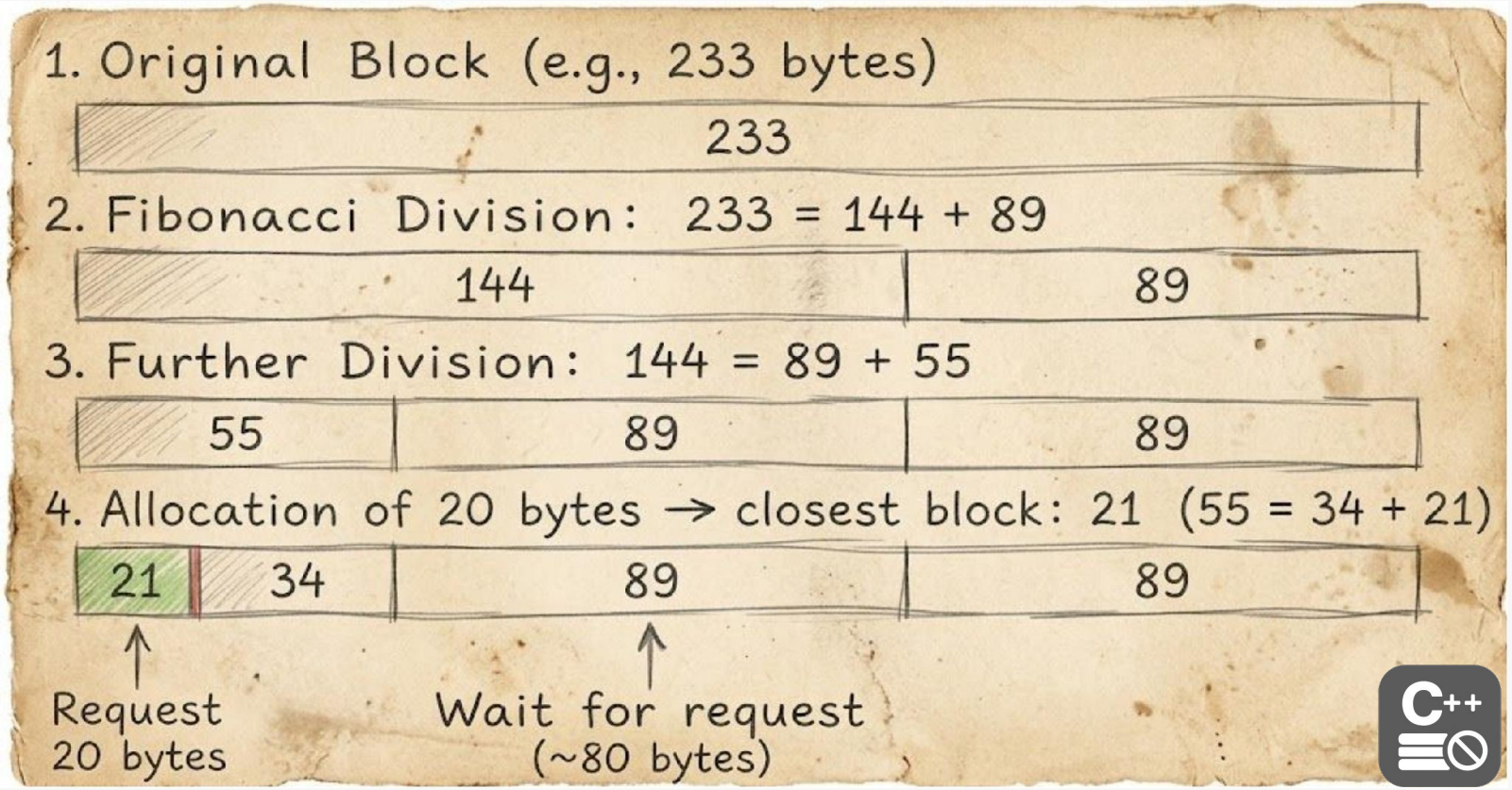
Parallel Allocation

N threads allocate simultaneously with no serialization bottleneck.

Linear Scalability

Throughput grows proportionally with thread count — no diminishing returns from lock contention.

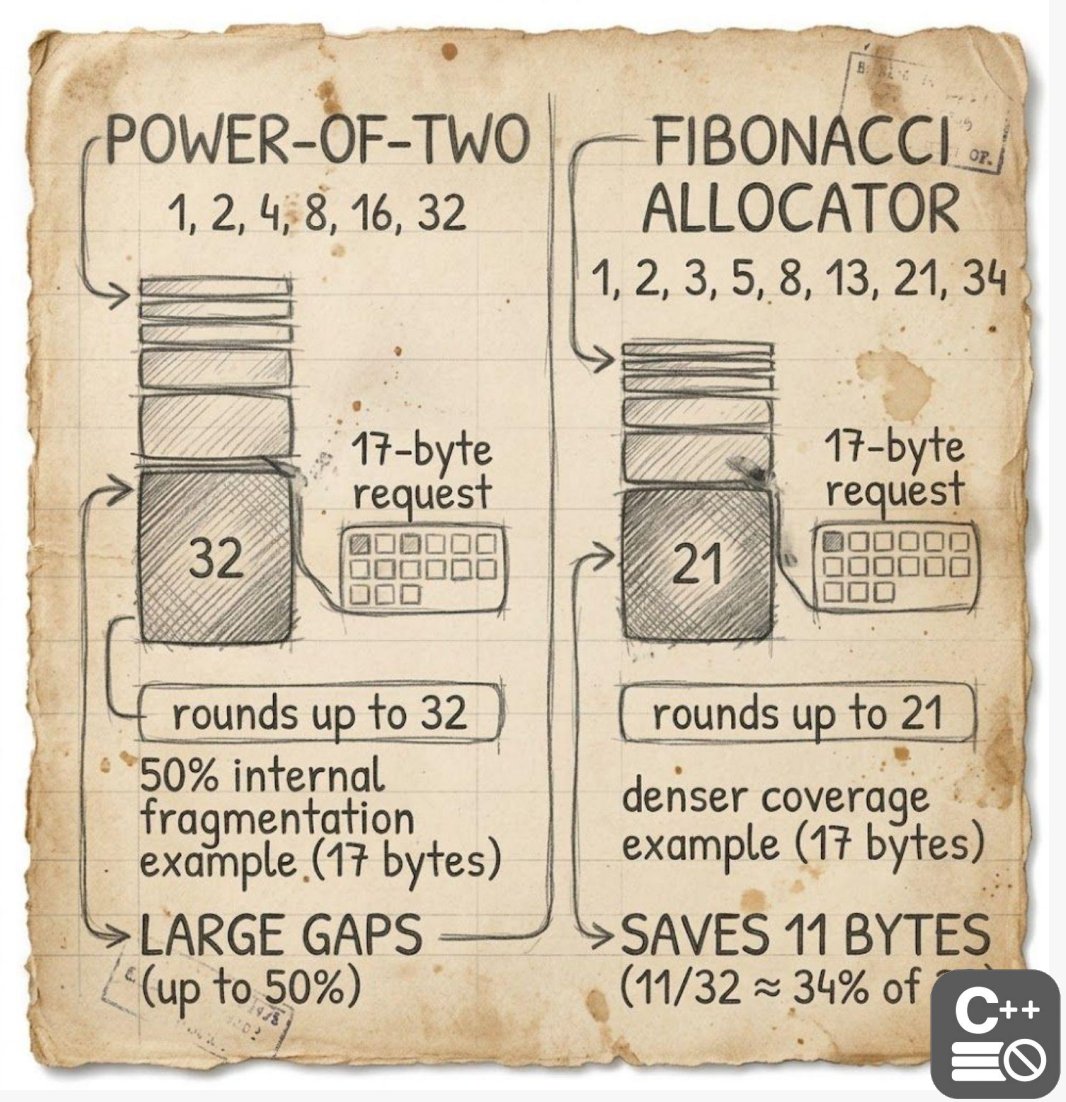
Fibonacci Allocator



Rather than rounding sizes to powers of two, the Fibonacci allocator bins blocks at Fibonacci-number boundaries: 1, 2, 3, 5, 8, 13, 21, 34, 55...

The ratio between consecutive Fibonacci numbers converges to $\varphi \approx 1.618$.

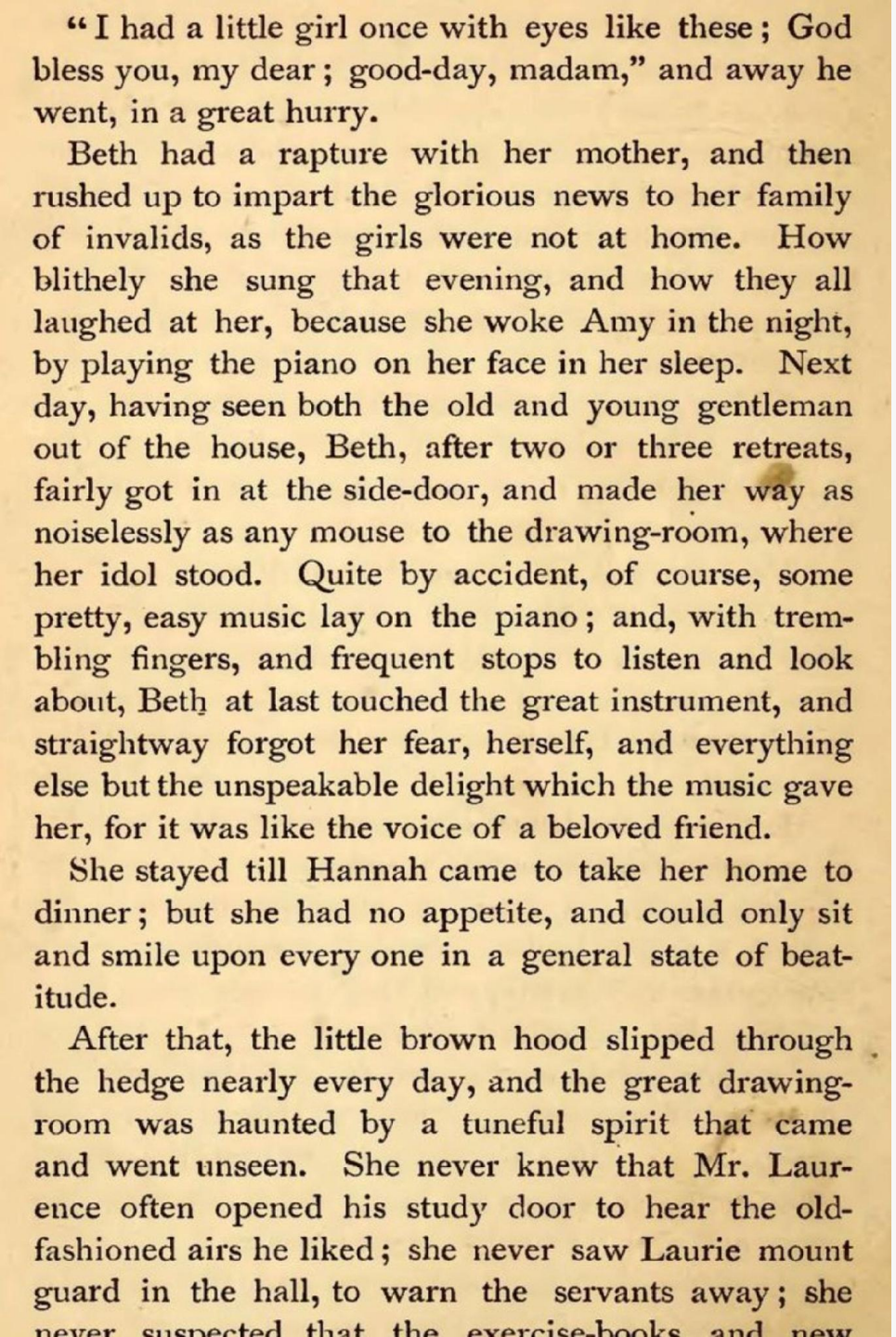
Power-of-Two vs. Fibonacci



The Fragmentation Gap

Power-of-two binning wastes up to **50% of allocated space on a single misaligned request. Fibonacci binning dramatically narrows the gap between adjacent size classes.**

This is especially valuable for variable-length data — natural-language strings, AST nodes, and parsed tokens — where sizes are unpredictable and rarely align to clean boundaries.



"I had a little girl once with eyes like these; God bless you, my dear; good-day, madam," and away he went, in a great hurry.

Beth had a rapture with her mother, and then rushed up to impart the glorious news to her family of invalids, as the girls were not at home. How blithely she sung that evening, and how they all laughed at her, because she woke Amy in the night, by playing the piano on her face in her sleep. Next day, having seen both the old and young gentleman out of the house, Beth, after two or three retreats, fairly got in at the side-door, and made her way as noiselessly as any mouse to the drawing-room, where her idol stood. Quite by accident, of course, some pretty, easy music lay on the piano; and, with trembling fingers, and frequent stops to listen and look about, Beth at last touched the great instrument, and straightway forgot her fear, herself, and everything else but the unspeakable delight which the music gave her, for it was like the voice of a beloved friend.

She stayed till Hannah came to take her home to dinner; but she had no appetite, and could only sit and smile upon every one in a general state of beatitude.

After that, the little brown hood slipped through the hedge nearly every day, and the great drawing-room was haunted by a tuneful spirit that came and went unseen. She never knew that Mr. Laurence often opened his study door to hear the old-fashioned airs he liked; she never saw Laurie mount guard in the hall, to warn the servants away; she never suspected that the exercise-books and new

Natural Fit for Language Workloads

Compilers, interpreters, and NLP pipelines allocate objects with inherently irregular sizes

Tokens, syntax nodes, string buffers.

Fibonacci binning matches this distribution far more closely than rigid power-of-two classes

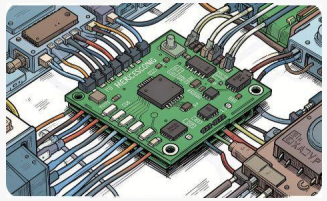
Reducing both memory waste and cache-line pollution from oversized blocks.

Better results for natural languages

String type (characters)	2*-block	Waste (bytes)	F*-block	Waste (bytes)
13 (keys)	16	3 (~23%)	13	0 (0%)
20 (names, variables)	32	12 (37%)	21	1 (~5%)
33 (timestamps)	64	31 (~48%)	34	1 (~2%)
50 (regular strings)	64	14 (~22%)	55	5 (~9%)
80 (code strings)	128	48 (~37%)	89	9 (~10%)
90 (error messages)	128	32 (~30%)	144	54 (~38%)

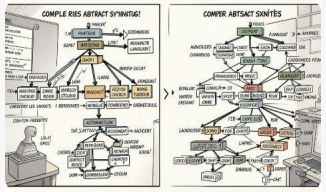


Fibonacci Allocator Uses



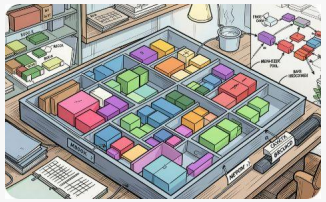
Memory-Constrained Systems

Embedded devices and microcontrollers where every byte of fragmentation directly impacts available RAM.



Parsers & Compilers

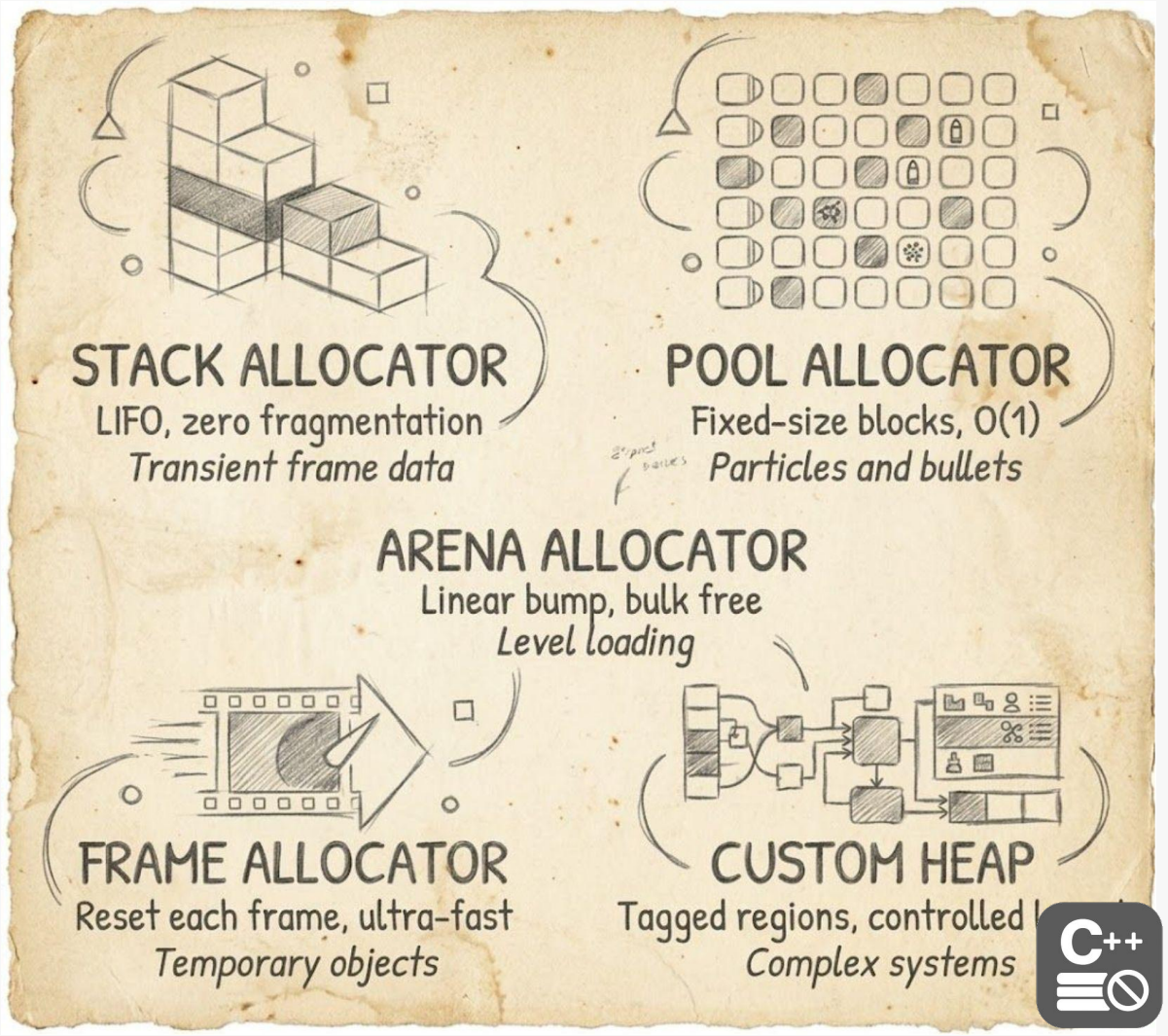
Natural-language parsers, compilers, and interpreters with irregular, non-power-of-two object sizes.



Specialized Object Pools

Pools managing heterogeneous object sizes without forcing artificial power-of-two alignment.

Allocators lives in game engines



Engines use a multiple allocators,
each serving a specific domain.
The key is knowing which pattern maps to which lifetime.

Not covered today, but very interesting

Performance-Oriented

- Compacting allocator — eliminates fragmentation by relocating live blocks
- TLSF — O(1) allocation via two-level size segregation
- Linear allocator with rollback — bump-pointer with checkpoint undo
- Arena allocator — bulk free by region, zero per-object overhead

Debug & Specialized

- Color-based / chaos allocator — poisoning patterns for corruption detection
- Lifetime-tracking allocator — monitors object lifespans for optimization
- Stack allocators — fixed, expandable, and outliving-the-stack variants
- Shared thread cache — hybrid between TLS and global heap



Chi sono :)









- 20+ years with C++
- 15+ years in game development
- shipped AA/AAA titles (Forgotten Empires, 4A Games, Playrix, Arkane, Gaijin, EA)
- book about game engine optimization **(Game++)**
- **ru/en/it** blog about gamedev, optimizations and related things **(dalerank.github.io)**
- **C++ w/o allocs (stepik.org/a/255930)**
- **Next book here (github.com/dalerank/playful_programming_cpp)**
- love code, algs and old cars

Contacts

tg : dalerank

in : linkedin.com/in/dalerank

git: github.com/dalerank

web: dalerank.github.io

Grazie!